Design Verification Interview

ChipDev	https://chipdev.io/question-list
ChipVerify	https://www.chipverify.com/systemverilog/systemverilog-interview-questions-set-1
DV面经	https://www.1point3acres.com/bbs/thread-1167331-1-1.html
ForkJoin.io	https://forkjoin.in/interview-questions-systemverilog/interview-questions-systemverilog-home.html
HDLBits	https://hdlbits.01xz.net/wiki/Problem_sets
LeetCode	https://leetcode.com/problems/reverse-bits/description/

TODO LIST

- ~~CLA/Divider from class~~
- ~~Cache coherence protocol & FSM & 3 Types~~
- ~~FIFO & Asynchronous FIFO~~
- ~~Clock Domain Crossing~~
- ~~Round Robin Arbiter~~
- ~~Fibonacci~~
- ~~UVM Phase~~
- ~~Moore/Mealy Machine~~
- ~~AMBA Interconnect/AXI~~
- ~~#19 on ChipDev~~

~~Covergroup & Properties & Constraints, have to write asserts manually~~

~~Polymorphism / Inheritance~~

~~Fork-join~~

~~LRU~~

~~Sequencer Arbitration~~

Callback

Mailbox / Semaphore

SV DPI

Past Questions

IMC Challenge

Challenge 3: Assessment

In stock trading, exchanges help match buyers and sellers by publishing trade offers on their exchange feed. We need to monitor a specific exchange feed to descide if we want to trade.

Feed Structure

Each feed message has two parts:

- **HEADER**: 8 bits
- **PAYLOAD**: 24 bits



Message Transmission

A feed message is received over 2 cycles:

- First cycle: 8 bits of **HEADER** and 8 bits of **PAYLOAD**
- Second cycle: remaining 16 bits of **PAYLOAD**

Key Definitions

- *Feed Message*: Consists of 8-bit **HEADER** and 24-bit **PAYLOAD** fields as described above.
- *Sequence*: A set of 4 valid feed messages.
- *Match Payload*: A specifc 24-bit value that the **PAYLOAD** field of each feed message should be compared against.
- *Matching Sequence*: A set of 4 match payloads that must appear in a sequence in order for `out_trigger` to be valid

Task

Create a module to monitor incoming data and detect specific patterns in the **PAYLOAD** fields.
The module should:

1. Identify when a full *sequence* is seen.
2. Check if the *sequence* matches the predefined *matching sequence*.
3. Output the concatenated **HEADER** fields of each *feed message* in the *sequence*.

Matching Sequence to Detect

1. 0xDF0132
2. 0x4DE91B
3. 0x73AC06
4. 0xA1C47D

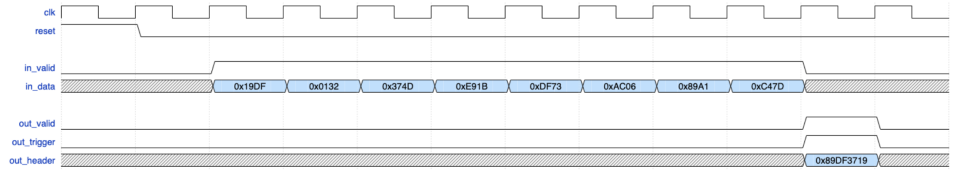
Signal Descriptions

Signal Name	Direction	Width (bits)	Description
`clk`	Input	1	System clock, all flip-flops are rising edge-triggered
`reset`	Input	1	Synchronous reset signal, active high
`in_valid`	Input	1	Indicates `in_data` is valid
`in_data`	Input	16	Incoming data containing **HEADER** and **PAYLOAD** fields
`out_valid`	Output	1	Indicates a full sequence has been seen, and `out_header` and `out_trigger` are valid
`out_trigger`	Output	1	Indicates **PAYLOAD** fields in the sequence matched the matching sequence
`out_header`	Output	32	Concatenated **HEADER** fields of the feed messages during a sequence

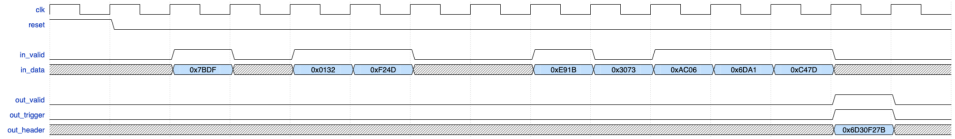
Waveforms

Below are some waveforms the implementation of your module should satisfy.

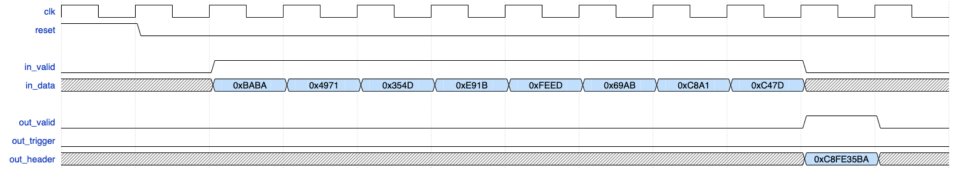
Waveform 1: *Sequence* that matches the *matching sequence*



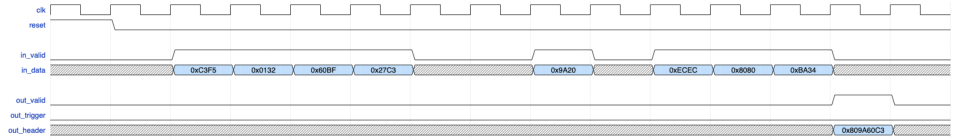
Waveform 2: *Sequence* that matches the *matching pattern* with `in_valid` gaps



Waveform 3: *Sequence* that does not match the *matching pattern*



Waveform 4: *Sequence* that does not match the matching sequence with `in_valid` gaps



You have the ability to view the waveform produced by your design. **We are only looking for your implementation to pass Simulation.**
You can also synthesize your design. Be aware this can take a few minutes to run.

```
// The following module you see before you receives a clock and
// has a synchronous reset. The reset is active high.
// Extend the module you have been given to drive out_valid,
// out_trigger, and out_header as described in the challenge
// description. You may add or remove code but please do NOT modify
// the module ports. The testbench provided will tell you when you
// have correctly done this. We are looking for the test to pass
// first and foremost.
// You have the ability to view or download the waveform produced
// by the design. You can also synthesize the design. Be aware
// this can take a few minutes to run.
module top (
 input logic clk,
 input logic reset,
 input logic in_valid,
 input logic [15:0] in_data,
 output logic out_valid,
 output logic out_trigger,
 output logic [31:0] out_header
);
 localparam logic [23:0] MATCH_DATA_1 = 24'hDF0132;
 localparam logic [23:0] MATCH_DATA_2 = 24'h4DE91B;
 localparam logic [23:0] MATCH_DATA_3 = 24'h73AC06;
 localparam logic [23:0] MATCH_DATA_4 = 24'hA1C47D;
 // We have incoming data packets with 32 bits each. I notices
something here:
 // 1. If we have matching sequence, out_triggered is HIGH,
out_valid is HIGH,
 //    the out_header follows {NEWEST,...,OLDEST} order.

 typedef enum [2:0] {IDLE, MATCH1, MATCH2, MATCH3, MATCH4} state_t;
 logic [15:0] in_data_d;
 reg [31:0] in_data_reg [0:3];
 logic [31:0] tmp_header;
 logic [7:0] counter;

 state_t state;

 always @(posedge clk) begin
 if(reset) begin
 state <= IDLE;
 counter <= 0;
```

Apple Formal Verification

Round 1 / Hiring manager

```
property handshake;
 $rose(req) |=> req until_with $rose(ack) ; // This property should be able to describe the handshake and fail at cycle C.
endproperty
```

Apple Design Verification (RTL Integration)

Round 1 / Hiring manager

Reset and CDC

1. Explain the difference between synchronous reset and asynchronous reset. What are the pros, cons, and typical use cases of each?
1. For CDC, explain how you would synchronize:
    - a single-bit signal
    - a pulse signal
    - a multi-bit signal
1. Compare common CDC methods such as 2-flop synchronizer, pulse synchronizer, enable-based synchronization, and asynchronous FIFO. When would you use each one?
1. Draw the circuit for a 2-flop synchronizer and explain why it reduces metastability risk.
1. For a multi-bit CDC crossing, when is an enable-based synchronizer sufficient, and when do you need an async FIFO?

FIFO / throughput / bandwidth

1. Suppose data is transferred from one clock domain to another with:
    - write side: 10 MHz, 2 data words every 10 cycles
    - read side: 1 MHz, 1 data word every cycle

    Can you determine the FIFO depth required?
1. If FIFO depth cannot be determined directly from only those numbers, what additional assumptions or constraints do you need?
1. Explain how you would size an asynchronous FIFO given source rate, destination rate, burstiness, and clock frequency mismatch.

Case – 10 : Specifications can be given in a different way. (Understanding the specifications is important here)

Given the following FIFO rules, how deep does the FIFO need to be to prevent underflow or overflow?

Frequency (clk A) = frequency (clk B)/4
Period (en_B) = period (clk_A)*100
Duty cycle (en_B) = 25%

Sol :

- ✓ Assume some numerical values, if the specifications are in this way.
- ✓ Assume frequency of clk_B = 100MHz
- ✓ So, the frequency of clk_A = 100MHz /4 = 25MHz.
- ✓ In the specifications given, the burst length is specified indirectly. The burst length is 100.
- ✓ Time required to write one data item = $\frac{1}{25\ MHz}$ = 40 nSec.
- ✓ Time required to write all the data in the burst = 100 * 40 nSec. = 4000 nSec.
- ✓ And the duty cycle of en_B is 25 % means that, out of 4000 nSec in which the writing process is completed, reading is done only in a period of 1000 nSec (25% of 4000 nSec.)
- ✓ So, the FIFO should be capable of holding the data which is being written in the remaining 3000 nSec.
- ✓ The no. of data items can be read in a period of 3000 nSec = $\left(\frac{3000\ nSec}{40\ nSec}\right) = 75$

So, the minimum depth of the FIFO should be 75.

Digital logic / circuits

1. Given two 3-bit inputs A and B, design logic to detect:
   - whether A == B
   - whether A > B

Draw the logic circuit, and derive the Boolean expressions, optionally using K-map minimization.

Panel round — interviewer 1

Serial memory protocol / coding

1. Given the following serial memory write protocol, implement the logic or FSM:

WRITE

- Din: A3-A2-A1-A0-R/W-WD3-WD2-WD1-WD0-a3-a2
- Dout: XX-XX-XX-XX-XXX-XX-XX-XX-XX-XX

Reconstruct the write transaction and design the corresponding serializer/deserializer or controller.

1. Given the following serial memory read protocol, implement the logic or FSM:

READ

- Din: A3-A2-A1-A0-R/W-XX-XX-XX-XX-XX
- Dout: XX-XX-XX-XX-XXX-RD3-RD2-RD1-RD0

Explain the timing, state transitions, and how data is captured/output.

Glitch handling

1. If a signal has a glitch, what would you do?
1. How would you solve a glitch problem outside the chip?
1. How would you solve a glitch problem inside the chip using digital circuits?

Panel round — interviewer 2

Power / low power design

1. What is a power domain?
1. In a multi-voltage design with different VDDs, why do you need a level shifter?
1. What is a clamp cell / isolation cell? Why is it needed to prevent X propagation during power gating?
1. Explain ICG clock gating. Draw a typical integrated clock gating circuit.
1. Why is clock gating often sensitive to a particular clock phase or edge?
1. Detect whether an input value is a power of 2. Can you relate that to one-hot detection?

Panel round — interviewer 3

STA / timing

1. In STA, how do you determine where timing is failing?
1. If a path fails timing, when can you fix functionality by inserting flops or pipeline stages?
1. What is the impact of adding flops on latency, throughput, and timing closure?

Cache architecture

1. In a cache, how do you evict a cache line?
1. Can you evict different blocks?
1. Can you evict only different bytes within a cache block, or is eviction done at cache-line granularity?
1. Explain cache line, block, byte offset, and eviction policy.

Arbiter

1. Design an arbiter such that:
   - if the current request is 111, grant the LSB requester in the current cycle
   - then grant requester 2 in one of the following two cycles, regardless of the request pattern at that moment

   Explain the arbitration rule and design the control logic/FSM.

Panel round — interviewer 4

One-hot / small logic coding

1. Given a 4-bit input, detect whether it is one-hot encoded.
1. What kind of hardware would synthesis generate for one-hot detection?

Counter design / optimization

1. Design a 2-bit counter that counts in the sequence:
   - 0 -> 1 -> 2 -> 0 -> 1 -> 2 -> ...

   or equivalently states like 00, 01, 10 / possibly noted as 012, 012...
1. Optimize that counter design for minimum gate count.
1. What is the smallest implementation you can achieve? The note suggests something like: “at minimum only two flops plus one combinational logic gate.”

Reset synchronizer / metastability

1. In a reset synchronizer for asynchronous-assert, synchronous-deassert reset, there are two flops.
1. The first flop may go metastable. Why does the second flop help?
1. What if the reset seen by the second flop also becomes metastable? What happens, why, and how serious is it?

Panel round — interviewer 5

Layout / devices / analog basics

1. Draw the NMOS and PMOS layout.
1. Draw an inverter layout.
1. Explain basic CMOS layout considerations.

ADC

1. For an ADC, how would you measure or derive a 100 MHz-related performance metric?

   The note is abbreviated here; this could mean:

   - how to measure a 100 MHz sampling ADC, or
   - how to derive sampling/output timing for a 100 MHz ADC.

   This item is somewhat unclear in the screenshot.

Cache comparison

1. Compare three kinds of cache organizations. Which one has what advantages?

   This likely refers to:

   - direct-mapped
   - set-associative
   - fully associative

   and asking for tradeoffs in hit rate, complexity, speed, and area.

Additional coding questions from page 2

1. A 16-bit value is incremented 1024 times over 1024 cycles. What should the output width be?
1. Count the number of 1s in a 1024-bit signal. What should the output width be?

Apple Design Verification (Cellular SoC)

1. Write a sequence detection with drawing FSM

   Similar to this question.

```
module model (
 input clk,
 input resetn,
 input din,
 output logic dout
);
typedef enum logic [2:0] {IDLE, MATCH1, MATCH2, MATCH3, MATCH4} state_t;
state_t state;
always @(posedge clk) begin
 if (!resetn) begin
 state <= IDLE;
 end else begin
 case (state)
 IDLE: state <= (din == 1) ? MATCH1 : IDLE;
 MATCH1: state <= (din == 0) ? MATCH2 : MATCH1;
 MATCH2: state <= (din == 1) ? MATCH3 : IDLE;
 MATCH3: state <= (din == 0) ? MATCH4 : MATCH1;
 MATCH4: state <= (din == 1) ? MATCH3 : IDLE;
 endcase
 end
end
assign dout = (state == MATCH4);
endmodule
```

1. Write a Fibbonacci Sequence generator

   You need to write module inputs and outputs

```
module fib_dyn (
 input int n,
 output logic [31:0] result
);
 logic [31:0] fib_arr[];
 int i;
 always_comb begin
 if (n <= 1) begin
 result = n;
 end else begin
 fib_arr = new[n+1];
 fib_arr[0] = 0;
 fib_arr[1] = 1;
 for (i = 2; i <= n; i++) begin
 fib_arr[i] = fib_arr[i-1] + fib_arr[i-2];
 end
 result = fib_arr[n];
 end
 end
endmodule
```

Interviewer asks: Have you considered the area of the code, what if we have infinitely large N

Fix: Use `curr` and `prev`.

```
module fib #(
 parameter int N = 10
)(
 output logic [31:0] result
);
 logic [31:0] prev2, prev1, curr;
 int i;
 always_comb begin
 if (N <= 1) begin
 result = N;
 end else begin
 prev2 = 0;
 prev1 = 1;
 for (i = 2; i <= N; i++) begin
 curr = prev1 + prev2;
 prev2 = prev1;
 prev1 = curr;
 end
 result = prev1;
 end
 end
endmodule
```

Apple Design Verification (Cellular Baseband)

Coding Question:

1. Write a testbench module that generates 100MHz `clk` and an asynchronous `rst`.

```
`timescale 1ns/1ps
module tb;
 logic clk;
 logic reset;
 // 100 MHz clock → period = 10ns
 initial clk = 0;
 always #5 clk = ~clk; // Question asked here: What is the difference from always clk = #5 ~clk;
 // async reset
 initial begin
 // Question asked here: What if we put always #5 clk = ~clk; inside this initial block, right this line.
 reset = 1;
 #20;
 reset = 0;
 end
endmodule
```

1. Given logic [31:0] payload, write a reusable class to calculate their logic [31:0] parity.
    - Payload can be between 1:100 in amount.
    - parity[0] = payload[0][0] ^ payload[1][0] ^ … ^ payload[N][0].
    - User should be able to randomize the number of payloads.

```
class packet;
 rand bit [31:0] payloads[]; // dynamic array of payloads
 // Question asked here: What if we use a queue like: rand bit [31:0] payloads[$], any differences?
 bit [31:0] parity;
 constraint size_c {
 payloads.size() inside {[1:100]};
 }
 function void post_randomize();
 parity = '0;
 foreach (payloads[i]) begin
 parity ^= payloads[i]; // XOR across all payloads (bitwise)
 end
 endfunction

 // Question asked here: Can you write without foreach, using two for loops?
 for (int i = 0; i < 32; i++) begin
 for (int j = 0; j < payload.size(); j++) begin
 parity[i] = parity[i] ^ payloads[j][i];
 end
 end
endclass
```

1. UVM Question:

   How do sequences and drivers interact? Say if we have two forked sequences generated by user with

   ```
   fork
    s1.start() // s1 last for 10 cycles
    s2.start() // s2 last for 20 cycles
   join
   ```

   Is it possible to determine which packet start first? When will the packet finish processing, 20 cycles or 30 cycles? Does one process needs to wait until the other to finish?

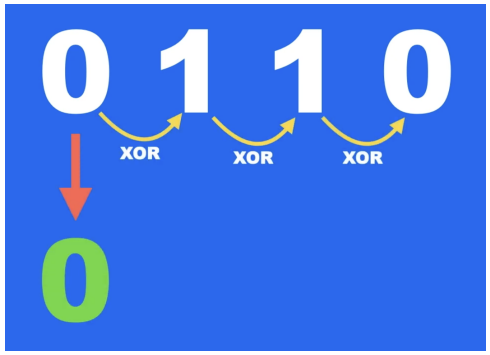
> ChatGPT answer:
>
> - s1.start() and s2.start() are launched concurrently by the fork...join.
> - If both sequences run on the same sequencer/driver, then only one item/transaction is driven at a time.
> - So:
>   - Which starts first at the sequence level? Both are started concurrently.
>   - Which gets to the driver first? Not deterministically guaranteed unless you set arbitration/priority.
> - Does one need to wait for the other sequence to completely finish?
>
>   No. It does not need to wait for the whole other sequence to finish. It only waits when competing for the sequencer/driver.
>
> - Total finish time: 20 or 30 cycles?
>   - If by “10 cycles” and “20 cycles” you mean each sequence holds the shared driver exclusively for that entire duration, then total is effectively 30 cycles.
>   - If they are just independent threads not contending for the same driver resource, then it could complete in 20 cycles.
>   - In the usual same sequencer/same driver case, answer is 30 cycles, not 20.
> - So the clean practical answer for normal UVM shared-driver case is:
>   - Order first: not guaranteed
>   - Total time: 30 cycles
>   - Wait behavior: they start together, but one waits for access to the shared sequencer/driver while the other is using it

SV Coding

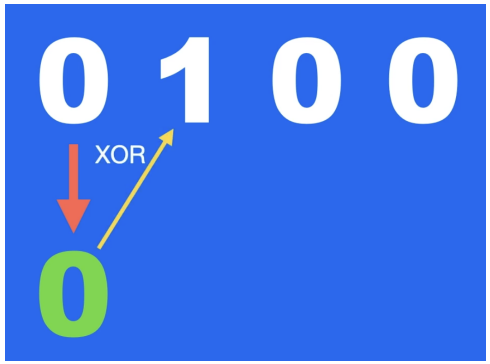
Gray Code

Used for error detection, MSB stays the same. Perform `XOR` operation.

Binary to Gray:



Gray to Binary:




```
module model #(parameter
 DATA_WIDTH = 4
) (
 input clk,
 input resetn,
 output logic [DATA_WIDTH-1:0] out
);
 logic [DATA_WIDTH-1:0] num, temp;
 int i;
 always @(posedge clk) begin
 if (!resetn) begin
 temp <= 0;
 num <= 1;
 end else begin
 num <= num + 1;
 for (i = 0; i < DATA_WIDTH - 1; i = i + 1) begin
 temp[i] <= num[i] ^ num[i+1];
 end
 temp[DATA_WIDTH-1] <= num[DATA_WIDTH-1];
 end
 end
 assign out = temp;
endmodule
```

Edge Detector

Write in two different ways

Register one cycle

```
module model (
 input clk,
 input resetn,
 input din,
 output dout
);
 logic din_d, pulse;
 always_ff @(posedge clk) begin
 if (!resetn) begin
 din_d <= 1'b0;
 pulse <= 1'b0;
 end else begin
 pulse <= din & ~din_d;
 din_d <= din;
 end
 end
 assign dout = pulse;
endmodule
```

Finite State Machine

```
module model (
 input clk,
 input resetn,
 input din,
 output dout
);
 typedef enum logic [1:0] {IDLE, PULSE, HIGH} state_t;
 state_t state;
 always_ff @(posedge clk) begin
 if (!resetn) begin
 state <= IDLE;
 end else begin
 case (state)
 IDLE: state <= din ? PULSE : IDLE;
 PULSE: state <= din ? HIGH : IDLE;
 HIGH: state <= din ? HIGH : IDLE;
 default: state <= IDLE;
 endcase
 end
 end
 assign dout = state == PULSE;
endmodule
```

What if we want same cycle pulse?

```
assign dout = resetn && din && (state == IDLE);
```

One-Hot

Smart Way

-1 and AND

```
module model #(parameter
 DATA_WIDTH = 32
) (
 input [DATA_WIDTH-1:0] din,
 output logic onehot
);
assign onehot = (din != 0) && ((din & (din - 1)) == 0);
endmodule
```

Traditional Way

Count and output if sum ==1

```
module model #(parameter
 DATA_WIDTH = 32
) (
 input [DATA_WIDTH-1:0] din,
 output logic onehot
);
 int i;
 logic [DATA_WIDTH-1:0] num_ones;
 always @(*) begin
 num_ones = 0;
 for (i=0; i < DATA_WIDTH; i++) begin
 num_ones += din[i];
 end
 end
 assign onehot = (num_ones == 1);
endmodule
```

Double Dabble (Binary → Decimal)

```
module model #(
 parameter DATA_WIDTH = 10
) (
 input logic [DATA_WIDTH-1:0] din,
 output logic [3:0] dec0,
 output logic [3:0] dec1,
 output logic [3:0] dec2
);
 logic [DATA_WIDTH+11:0] bcd;
 int i;
 always_comb begin
 bcd = '0;
 bcd[DATA_WIDTH-1:0] = din;
 for (i = 0; i < DATA_WIDTH; i++) begin
 if (bcd[DATA_WIDTH+3:DATA_WIDTH] >= 5)
 bcd[DATA_WIDTH+3:DATA_WIDTH] += 3;
 if (bcd[DATA_WIDTH+7:DATA_WIDTH+4] >= 5)
 bcd[DATA_WIDTH+7:DATA_WIDTH+4] += 3;
 if (bcd[DATA_WIDTH+11:DATA_WIDTH+8] >= 5)
 bcd[DATA_WIDTH+11:DATA_WIDTH+8] += 3;
 bcd = bcd << 1;
 end
 dec2 = bcd[DATA_WIDTH+11:DATA_WIDTH+8];
 dec1 = bcd[DATA_WIDTH+7:DATA_WIDTH+4];
 dec0 = bcd[DATA_WIDTH+3:DATA_WIDTH];
 end
endmodule
```

```
Number 21
Binary
(0001 0101)

                                                
```

```
module model #(parameter
 DATA_WIDTH=32
) (
 input [DATA_WIDTH-1:0] din,
 output logic dout
);
 logic is_palindrome;
 always_comb begin
 is_palindrome = 1;
 for (int i = 0; i < DATA_WIDTH/2; i = i + 1) begin
 is_palindrome = (din[i] == din[DATA_WIDTH-1-i]) && is_palindrome;
 end
 end
 assign dout = is_palindrome;
endmodule
```

Reversing Bits

Traditional way

```
logic [DATA_WIDTH-1:0] temp;
always_comb begin
 for (int i = 0; i < DATA_WIDTH; i = i + 1) begin
 temp[DATA_WIDTH-i-1] = din[i];
 end
end
assign dout = temp;
```

Faster way

```
assign dout = {<<{din}};
```

Clock Division

Division by EVEN

Example: Division by 2/4/6

```
module model (
 input clk,
 input resetn,
 output logic div2,
 output logic div4,
 output logic div6
);
 logic cnt2;
 logic [1:0] cnt4;
 logic [3:0] cnt6;
 always @(posedge clk) begin
 if (!resetn) begin
 cnt2 <= 0;
 cnt4 <= 0;
 cnt6 <= 0;
 end else begin
 cnt2 <= cnt2 + 1;
 cnt4 <= cnt4 + 1;
 cnt6 <= (cnt6 + 1) % 6;
 end
 end
 assign div2 = (cnt2 == 1);
 assign div4 = (cnt4 == 1) || (cnt4 == 2);
 assign div6 = (cnt6 == 1) || (cnt6 == 2) || (cnt6 == 3);
endmodule
```

Or, simply use 1 counter:

```
module model (
 input clk,
 input resetn,
 output logic div2,
 output logic div4,
 output logic div6
);
// Divide by even
logic [3:0] counter;
always @(posedge clk) begin
 if (!resetn) begin
 counter <= 11;
 end else begin
 counter <= (counter < 11) ? counter + 1 : 0;
 end
end
assign div2 = counter inside {4'd0, 4'd2, 4'd4, 4'd6, 4'd
8, 4'd10};
assign div4 = counter inside {4'd0, 4'd1, 4'd4, 4'd5, 4'd
8, 4'd9};
assign div6 = counter inside {4'd0, 4'd1, 4'd2, 4'd6, 4'd
7, 4'd8};
// Divide by odd
endmodule
```

Division by ODD

Sequence Detection

The `len` is used for corner case after reset.

Use camera-like patterns to capture the most recently received `n` bits.

```
module model (
 input clk,
 input resetn,
 input [4:0] init,
 input din,
 output logic seen
);
 logic [4:0] cur, len, target;
 logic reset_seen;
 always @(posedge clk) begin
 if (!resetn) begin
 reset_seen <= 1;
 cur <= 0;
 len <= 0;
 end else begin
 cur <= {cur[3:0], din};
 len <= (len < 5) ? len + 1 : len;
 end
 end
 assign target = (resetn && reset_seen && len == 0) ? init : target;
 assign seen = reset_seen && (cur == target) && (len == 5);
endmodule
```

Python Coding

206. Reverse Linked List

https://wkun-upenn.github.io/code-visualizer/reverse_linked_list.html

```
# Definition for singly-linked list.
# class ListNode(object):
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution(object):
def reverseList(self, head):
"""
        :type head: Optional[ListNode]
        :rtype: Optional[ListNode]
        """
curr = head
prev = None
while (curr):
nxt = curr.next
curr.next = prev
prev = curr
curr = nxt
return prev
```

215. K-th Largest Element

https://wkun-upenn.github.io/code-visualizer/kth_largest_heap.html

```
class Solution(object):
def findKthLargest(self, nums, k):
"""
        :type nums: List[int]
        :type k: int
        :rtype: int
        """
for i in range(len(nums)):
nums[i] = nums[i] * -1
heapq.heapify(nums)
for _ in range(k-1):
heapq.heappop(nums)
return nums[0]*-1
```

143. Reorder List

https://wkun-upenn.github.io/code-visualizer/reorder_list.html

```
# Definition for singly-linked list.
# class ListNode(object):
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution(object):
def reorderList(self, head):
"""
        :type head: Optional[ListNode]
        :rtype: None Do not return anything, modify head in-place
instead.
        """
slow, fast = head, head.next
while fast and fast.next:
slow = slow.next
fast = fast.next.next
second = slow.next
prev = slow.next = None
while second:
tmp = second.next
second.next = prev
prev = second
second = tmp
first, second = head, prev
while second:
tmp1, tmp2 = first.next, second.next
first.next = second
second.next = tmp1
first, second = tmp1, tmp2
```

UVM

Essential SV for UVM

Randomization

When a `rand` item is defined and we call `.randomize()`, we can define a `post_randomize()` function which executes automatically after each randomization call.

```
class randframe;
 local string name;
 rand local bit [3:0] length;
 bit [7:0] payload[];
 function new(string name);
 this.name = name;
 endfunction
 function void post_randomize();
 payload = new[length];
 foreach (payload[i]) payload[i] = $urandom;
 endfunction
endclass
randframe rframe = new("rframe");
int ok = rframe.randomize();
```

Inheritance and Polymorphism

UVM relies heavily on OOP. Key patterns:

- **Access modifiers**: `local` (class only), `protected` (class + subclasses), default (public within package)
- `this` refers to the current class instance
- **Virtual methods**: declared `virtual` in base class, overridden in subclass — dynamic dispatch at runtime
- **Pure virtual / abstract classes**: `pure virtual function void drive();` — forces subclasses to implement; base class cannot be instantiated directly
- **Parameterized classes**: `class my_class #(type T = uvm_sequence_item) ...;` — used for type-generic drivers/sequencers

```
// Base class declares virtual method
class base_driver extends uvm_driver #(my_packet);
 virtual task run_phase(uvm_phase phase);
 // base behavior
 endtask
endclass
// Subclass overrides — factory creates correct type at runtime
class fast_driver extends base_driver;
 virtual task run_phase(uvm_phase phase);
 // fast behavior
 endtask
endclass
// Polymorphic usage:
base_driver drv = fast_driver::type_id::create("drv", this);
drv.run_phase(phase); // calls fast_driver.run_phase()
```

Data Modeling

Basic Sequence Item

Steps: (1) extend `uvm_sequence_item`, (2) data constructor `(name)` + `super.new(name)`, (3) utility macro block to register with factory and enable field automation.

```
typedef enum bit {GOOD_PARITY, BAD_PARITY} parity_t;
class yapp_packet extends uvm_sequence_item;
 rand bit [5:0] length;
 rand bit [1:0] addr;
 rand bit [7:0] payload[];
 bit [7:0] parity;
 rand parity_t parity_type;
 rand int packet_delay;
 constraint payload_size { length == payload.size(); }
 constraint default_length { length > 0; length < 64; }
 constraint pkt_delay { packet_delay > 1; packet_delay < 20; }
 function new(string name = "yapp_packet");
 super.new(name);
 endfunction
 `uvm_object_utils_begin(yapp_packet)
 `uvm_field_int(length, UVM_ALL_ON)
 `uvm_field_array_int(payload, UVM_ALL_ON)
 `uvm_field_int(parity, UVM_ALL_ON + UVM_BIN)
 `uvm_field_enum(parity_t, parity_type, UVM_ALL_ON)
 `uvm_field_int(packet_delay, UVM_ALL_ON + UVM_NOCOMPARE)
 `uvm_object_utils_end
endclass : yapp_packet
```

UVM Field Macros

Syntax: `uvm_field_*(<field_name>, <flags>)` — enables copy, compare, print, record, pack/unpack.

Non-array: `uvm_field_int ,` uvm_field_object , `uvm_field_string ,` uvm_field_real , `uvm_field_enum(enum_type, field, flags)`

Dynamic arrays: `uvm_field_array_int/object/string/enum`

Static arrays: `uvm_field_sarray_int/object/string/enum`

Queues: `uvm_field_queue_int/object/string/enum`

Associative arrays: `uvm_field_aa_int_string ,` uvm_field_aa_object_string , `uvm_field_aa_int_int` ` etc.

Flag Arguments:

- `UVM_ALL_ON` — enable all operations (copy, compare, print, record, pack)
- `UVM_NOCOMPARE` — exclude from compare
- `UVM_NOCOPY` — exclude from copy/clone
- `UVM_NOPRINT` — exclude from print
- `UVM_NOPACK` — exclude from pack/unpack
- `UVM_NORECORD` — exclude from transaction recording
- Radix flags (print display): `UVM_BIN`, `UVM_DEC`, `UVM_HEX`, `UVM_OCT`, `UVM_UNSIGNED`

Copy and Clone

- `p2.copy(p1)` — copies all fields; both objects must already exist; does not copy name
- `$cast(p3, p1.clone())` — creates new instance, copies all fields including name; returns `uvm_object` so must `$cast`

Comparison and Print

- `p2.compare(p1)` — field-level comparison using automated fields
- `p1.print()` — table view (default); `p1.print(uvm_default_tree_printer)` for tree view
- `str = p1.sprint()` — creates string without printing

Simulation Phases

All phase methods take single `uvm_phase phase` argument. All phases except `run_phase` are **functions** (zero-time). `run_phase` is a **task** (consumes time). `build_phase` MUST call `super.build_phase(phase)` first.

Phase Order

`build → connect → end_of_elaboration → start_of_simulation → run → extract → check → report → final`

Run-Time Sub-Phases

Sub-phases run in parallel with `run_phase`, executing in order:

`pre_reset → reset → post_reset → pre_configure → configure → post_configure → pre_main → main → post_main → pre_shutdown → shutdown → post_shutdown`

A UVC can run stimulus in `reset_phase` while another runs in `main_phase` — they overlap with `run_phase`. The phase scheduler waits for all UVCs to complete a sub-phase before advancing.

```
virtual function void build_phase(uvm_phase phase);
 super.build_phase(phase); // MUST be first — applies config settings
endfunction
virtual task run_phase(uvm_phase phase); // task, not function
 forever begin
 seq_item_port.get_next_item(req);
 drive_packet(req);
 seq_item_port.item_done();
 end
endtask
```

Test and Testbench Class

Components vs Objects

- **Components** (`uvm_component` subclasses): persistent, have parent pointer, part of hierarchy. Constructor: `new(name, parent)`. Examples: agents, drivers, monitors, env, test.
- **Objects** (`uvm_object` subclasses): transient, no parent. Constructor: `new(name)`. Examples: sequence items, sequences, config objects, register models.

All instances should be created via factory: `<class>::type_id::create("name", parent_or_null)`.

Component Template

```
class my_component extends <uvm_component_subclass>;
 `uvm_component_utils(my_component)
 // OR with field macros:
 `uvm_component_utils_begin(my_component)
 `uvm_field_int(my_prop, UVM_ALL_ON)
 `uvm_component_utils_end
 function new(string name, uvm_component parent);
 super.new(name, parent);
 endfunction
endclass
```

Activating Phases: run_test()

```
// In top-level module:
initial begin
 run_test("my_test_name");
 // Or via plusarg: +UVM_TESTNAME=my_test_name
end
```

Report Macros

```
`uvm_info(id, message, verbosity) // informational
`uvm_warning(id, message) // warning
`uvm_error(id, message) // non-fatal error
`uvm_fatal(id, message) // fatal — stops simulation
// Verbosity levels (low to high):
// UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
// Best practices:
`uvm_info(get_type_name(), $sformatf("Packet received:\n%s", pkt.sprint()), UVM_LOW)
```

Interface UVC

UVC Structure

UVC → Agent → Sequencer + Driver + Monitor

Only **active** components (sequencer, driver) need a **type parameter** — they directly handle sequence items.

Active flow: Sequence creates packet → sequencer → driver gets packet via TLM → driver drives DUT pins via virtual interface.

Agent with is_active Configuration

```
class yapp_agent extends uvm_agent;
 `uvm_component_utils_begin(yapp_agent)
 `uvm_field_enum(uvm_active_passive_enum, is_active, UVM_ALL_ON)
 `uvm_component_utils_end
 yapp_sequencer sequencer;
 yapp_driver driver;
 yapp_monitor monitor;
 virtual function void build_phase(uvm_phase phase);
 super.build_phase(phase); // is_active config applied here
 monitor = yapp_monitor::type_id::create("monitor", this);
 if (is_active == UVM_ACTIVE) begin
 sequencer = yapp_sequencer::type_id::create("sequencer", this);
 driver = yapp_driver::type_id::create("driver", this);
 end
 endfunction
 virtual function void connect_phase(uvm_phase phase);
 if (is_active == UVM_ACTIVE)
 driver.seq_item_port.connect(sequencer.seq_item_export);
 endfunction
endclass
```

Sequence-Driver TLM Connection

Driver calls `seq_item_port.get_next_item(req)` to request next item, then `seq_item_port.item_done()` when finished. This blocking get/done handshake paces the sequence generation.

UVC Package File Include Order

packet → monitor → driver → sequencer → sequences → agent → env

Configuration (uvm_config_db)

Setting Configuration

```
// General syntax:
uvm_config_db#(T)::set(context, inst_name, field_name, value);
// Examples — set is_active to PASSIVE for a specific agent:
uvm_config_db#(uvm_bitstream_t)::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
// Shorthand typedef:
uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
// Wildcard instance paths (wildcards allowed in inst_name only):
uvm_config_int::set(this, "tb.env.agent?", "is_active", UVM_PASSIVE); // ?=single char
uvm_config_int::set(this, "*agent*", "is_active", UVM_ACTIVE); // *=any
uvm_config_int::set(this, "*", "recording_detail", 1); // all instances
```

Automatic Config Application

When `super.build_phase(phase)` is called, it calls `apply_config_settings()` automatically, which matches config DB entries to the component's **automated** properties (field macro required).

Config fails silently if: property not automated, instance path typo, type mismatch, or wildcards used in field_name.

Config Setting Rules

- Config properties **must be automated** with a field macro
- Only `inst_name` argument supports `*` and `?` wildcards
- Higher scope settings take **precedence** (test > testbench > env)
- Same scope: **last one in wins**

Debugging Configuration

```
// Call in check_phase — reports settings that were SET but never matched:
function void check_phase(uvm_phase phase);
 check_config_usage();
endfunction
// Output: lists field_name, pathname (regex), type, value for unmatched settings
```

uvm_config_db Methods

- `set(cntxt, inst_name, field_name, value)` — write entry
- `get(cntxt, inst_name, field_name, ref value)` — read entry, returns 1 if found
- `exists(cntxt, inst_name, field_name, spell_chk=0)` — check existence
- `wait_modified(cntxt, inst_name, field_name)` — block until setting changes

Explicit get (Required for top-module set or non-build-phase)

```
// get must be explicitly called when set came from top module:
if (!uvm_config_db#(virtual yapp_if)::get(this, "", "vif", vif))
 `uvm_error("NOVIF", "Missing virtual interface")
```

Configuration Object Pattern

```
class hbus_cfg_obj extends uvm_object;
 int num_masters = 1;
 `uvm_object_utils_begin(hbus_cfg_obj)
 `uvm_field_int(num_masters, UVM_ALL_ON)
 `uvm_object_utils_end
endclass
// Set from test:
uvm_config_db#(hbus_cfg_obj)::set(this, "tb.hbus", "hbus_cfg", cfg_obj);
// In hbus_env build_phase — automatic match via super.build_phase:
if (hbus_cfg == null) begin
 `uvm_warning("HBUS_CFG", "No config found — using default")
 hbus_cfg = hbus_cfg_obj::type_id::create("hbus_cfg", this);
end
```

Factory Override

Factory Concept

Central registry for creating class instances. Replace `new()` with `create()` everywhere so type substitution can be applied without modifying original code.

Factory Create Syntax

```
// Components (has parent):
driver = yapp_driver::type_id::create("driver", this);
// Data items / sequences (no parent — null default):
packet = yapp_packet::type_id::create("packet");
// Note: TLM connection objects use new(), not create()
yapp_out = new("yapp_out", this);
```

Type Override (all instances)

```
// In build_phase of test, BEFORE super.build_phase:
set_type_override_by_type(yapp_packet::get_type(), short_packet::get_type());
// Every create for yapp_packet now returns short_packet
class short_packet extends yapp_packet;
 `uvm_object_utils(short_packet) // must have own utility macro
 constraint short_length_const { length < 20; }
endclass
```

Instance Override (specific path)

```
set_inst_override_by_type("env1.agent0.*",
 yapp_packet::get_type(), short_packet::get_type());
```

Override Rules

1. **Instance overrides take precedence** over type overrides
1. Multiple instance overrides: **first match** applies
1. UVM 1.2: type override of A→A **resets** (removes) the override on A

Debugging Factory

```
// UVM 1.1:
factory.print();
// UVM 1.2:
uvm_factory my_f = uvm_factory::get();
my_f.print(); // 0=overrides only, 1=all user types (default), 2=include UVM types
```

Sequences

Sequence Structure

Sequences are **data objects** (not components): `uvm_object_utils`, data constructor `(name)`, stimulus defined in `body()` task.

```
class yapp_seq extends uvm_sequence #(yapp_packet);
 // Type param defines built-in handle: yapp_packet req;
 `uvm_object_utils(yapp_seq)
 function new(string name = "yapp_seq");
 super.new(name);
 endfunction
 virtual task body();
 `uvm_info(get_type_name(), "Call yapp_seq", UVM_LOW)
 `uvm_do(req) // 1 random packet
 `uvm_do_with(req, {addr == 0;}) // 1 packet constrained to addr=0
 endtask
endclass
```

`uvm_do Macro — 7 Steps

Step 1: factory create → Step 2: synchronize with sequencer (start_item) → Step 3: pre_do() hook → Step 4: randomize → Step 5: mid_do() hook → Step 6: send item, wait item_done (finish_item) → Step 7: post_do() hook

Macro variants:

- `uvm_do(req)` — all 7 steps
- `uvm_do_with(req, {addr==0;})` — all 7 steps + inline constraints
- `uvm_create(req)` — step 1 only (factory create)
- `uvm_send(req)` — steps 2–7, skip randomization
- `uvm_rand_send(req)` — steps 2–7 with randomization
- `uvm_rand_send_with(req, {constraints})` — steps 2–7 + constraints
- `uvm_do_on(seq, sequencer)` — run sequence on specific sequencer (multichannel)

Explicit Flow (alternative to macros)

```
virtual task body();
 req = yapp_packet::type_id::create("req"); // no 'this' for data items
 start_item(req); // steps 2-3
 assert(req.randomize() with {addr == 0;});
 finish_item(req); // steps 5-7
endtask
```

Directed Stimulus with Create/Send

```
virtual task body();
 `uvm_create(req) // factory create
 assert(req.randomize()); // randomize for legal packet
 req.addr = 3; // overwrite specific field
 req.set_parity(); // housekeeping
 `uvm_send(req) // send without re-randomizing
endtask
```

Sequence Properties and Nesting

```
class yapp_seq_addr extends uvm_sequence #(yapp_packet);
 rand bit [1:0] seqadd;
 constraint c1 { seqadd <= 2'b01; } // prevent illegal addr 3
 task body();
 `uvm_do_with(req, {addr == seqadd;})
 `uvm_do_with(req, {addr == seqadd + 1;})
 endtask
endclass
class nested extends uvm_sequence #(yapp_packet);
 yapp_seq_addr ysa; // handle for sub-sequence
 task body();
 `uvm_do(req)
 `uvm_do_with(ysa, {seqadd == 0;}) // execute sub-sequence with constraint
 `uvm_do(req)
 endtask
endclass
```

Executing Sequences

```
// Method 1: Default sequence via config_db (in test build_phase):
uvm_config_wrapper::set(this,
 "tb.yapp.agent.sequencer.run_phase",
 "default_sequence",
 yapp_012_seq::get_type());
// Method 2: Direct start from test run_phase:
yapp_seqr = tb.yapp.agent.sequencer; // assign in connect_phase
task run_phase(uvm_phase phase);
 phase.raise_objection(this, "mytest");
 ok = yapp_addr.randomize(); // if seq has rand props, must randomize explicitly
 yapp_addr.start(yapp_seqr);
 phase.drop_objection(this, "mytest");
endtask
```

Objection Handling

Objections prevent the `run_phase` from ending. Raise at start of stimulus, drop when done.

```
// In test (when calling start()):
task run_phase(uvm_phase phase);
 phase.raise_objection(this);
 my_seq.start(seqr);
 phase.drop_objection(this);
endtask
// In sequence (when used as default_sequence):
task body();
 if (starting_phase != null) // null means called from test
 starting_phase.raise_objection(this, get_type_name());
 // ... stimulus ...
 if (starting_phase != null)
 starting_phase.drop_objection(this, get_type_name());
endtask
// Subsequences should NOT handle objections
```

Objection via pre/post_body (base sequence pattern)

```
class base_seq extends uvm_sequence #(yapp_packet);
 virtual task pre_body();
 if (starting_phase != null)
 starting_phase.raise_objection(this, get_type_name());
 endtask
 virtual task post_body();
 if (starting_phase != null)
 starting_phase.drop_objection(this, get_type_name());
 endtask
 // pre/post_body NOT called for subsequences — objection-safe!
endclass
```

Drain Time

```
task run_phase(uvm_phase phase);
 uvm_objection obj = phase.get_objection();
 obj.set_drain_time(this, 200ns); // wait 200ns after last objection drops
endtask
```

Virtual Interface

Why Virtual Interfaces

UVM class components cannot directly connect to SV interface instances (interfaces are static). Virtual interfaces are interface variables that hold references to interface instances and can be declared as class properties.

Setting Virtual Interface (from top module)

```
module tb_top;
 yapp_if in0(clk, reset); // interface instance
 initial begin
 // Context = null (calling from module, not class instance)
 uvm_config_db#(virtual interface yapp_if)::set(
 null,
 "uvm_test_top.tb.yapp.*", // path to all UVC subcomponents
 "vif",
 hw_top.in0); // absolute hierarchical path to interface
 run_test();
 end
endmodule
```

Getting Virtual Interface (in driver/monitor)

```
class yapp_tx_driver extends uvm_driver #(yapp_packet);
 virtual interface yapp_if vif;
 function void connect_phase(uvm_phase phase);
 // Explicit get is ALWAYS required when set came from top module:
 if (!uvm_config_db#(virtual interface yapp_if)::get(this, "", "vif", vif))
 `uvm_error("NOVIF", "Missing virtual interface")
 endfunction
endclass
```

Convenience typedef (recommended)

```
// In UVC package — visible to both module and class files:
typedef uvm_config_db#(virtual interface yapp_if) yapp_vif_cfg;
// Top module:
yapp_vif_cfg::set(null, "*.tb.yapp.*", "vif", in0);
// Driver/monitor:
if (!yapp_vif_cfg::get(this, "", "vif", vif)) `uvm_error("NOVIF", "")
```

Virtual Interface Limitations

```
// Virtual interfaces cannot drive nets (wire) — procedural assignment to net is illegal
// Solution for bidirectional (inout) DUT ports:
interface hbus_if;
 wire [7:0] hdata_w; // map to DUT inout port; read via VIF
 logic [7:0] hdata; // write via VIF (procedural assignment to logic is OK)
 assign hdata_w = hdata; // drive wire from logic
endinterface
```

Multiple UVCs and Multichannel Sequencer

Multi-Agent Interface UVCs

An interface UVC can have multiple agents (TX and RX). Benefits:

- Develop and test UVC standalone before DUT is available (TX drives, RX samples same bus)
- Reuse: same UVC used by multiple design teams (TX for one team's input, RX for another's output)
- `has_tx` / `has_rx` config properties control which agents are built

```
class my_uvc extends uvm_env;
 bit has_tx = 1;
 bit has_rx = 1;
 tx_agent tx;
 rx_agent rx;
 bus_monitor mon; // one bus-level monitor shared by all agents
 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 mon = bus_monitor::type_id::create("mon", this);
 if (has_tx) tx = tx_agent::type_id::create("tx", this);
 if (has_rx) rx = rx_agent::type_id::create("rx", this);
 endfunction
endclass
```

Multichannel (Virtual) Sequencer

Coordinates stimulus across multiple UVC sequencers. Created in testbench; holds handles (references) to UVC sequencer instances.

```
class router_mcsequencer extends uvm_sequencer; // no type param
 `uvm_component_utils(router_mcsequencer)
 yapp_tx_sequencer yapp_seqr;
 hbus_master_sequencer hbus_seqr;
endclass
```

p_sequencer — Access Sequencer Properties from Sequence

```
class router_mcseq extends uvm_sequence; // no type parameter
 `uvm_object_utils(router_mcseq)
 `uvm_declare_p_sequencer(router_mcsequencer)
 // Declares: router_mcsequencer p_sequencer;
 // Link made when sequence is executed on the sequencer
 hbus_small_packet_seq h_small;
 yapp_seq_addr y_a;
 virtual task body();
 `uvm_do_on(h_small, p_sequencer.hbus_seqr)
 `uvm_do_on_with(y_a, p_sequencer.yapp_seqr, {y_a.seqadd == 1;})
 endtask
endclass
```

Instantiating Multichannel Sequencer in Testbench

```
class router_tb extends uvm_env;
 router_mcsequencer mcseqr;
 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 mcseqr = router_mcsequencer::type_id::create("mcseqr", this);
 endfunction
 function void connect_phase(uvm_phase phase);
 // Assign actual UVC sequencer instances to multichannel references:
 mcseqr.hbus_seqr = hbus.masters[0].sequencer; // handle copy, not string path
 mcseqr.yapp_seqr = yapp.tx_agent.sequencer;
 endfunction
endclass
```

Setting the Multichannel Default Sequence

```
// In test build_phase:
uvm_config_wrapper::set(this, "tb.mcseqr.run_phase",
 "default_sequence", router_mcseq::get_type());
// UVCs NOT controlled by multichannel sequencer still need sequences:
uvm_config_wrapper::set(this, "tb.chan?.*.sequencer.run_phase",
 "default_sequence", channel_rx_resp_seq::get_type());
```

Scoreboard

Scoreboard Architecture

1. **Transfer function (reference model)** — mimics DUT behavior, computes expected output from input
1. **Expected data store** — queue (in-order) or associative array (out-of-order)
1. **Checking logic** — compares actual DUT output vs expected

Analysis Interface (uvm_analysis_port / uvm_analysis_imp)

Broadcast capability: one port can connect to zero, one, or **many** consumers (unique to analysis).

```
// In monitor (producer):
uvm_analysis_port #(yapp_packet) yapp_out = new("yapp_out", this);
// ...
yapp_out.write(collected_packet); // broadcasts to all connected imps
// In scoreboard (consumer):
uvm_analysis_imp #(yapp_packet, router_sb) yapp_in = new("yapp_in", this);
function void write(input yapp_packet pkt);
 case (pkt.addr)
 2'b00: q0.push_back(pkt);
 2'b01: q1.push_back(pkt);
 2'b10: q2.push_back(pkt);
 endcase
endfunction
// In testbench connect_phase:
yapp.agent.monitor.yapp_out.connect(scoreboard.yapp_in);
```

Note: TLM connection objects use `new()`, NOT `create()` — they are tightly bound to specific method names and should not be factory-overridden.

Analysis FIFO (Decoupled Scoreboard)

`uvm_tlm_analysis_fifo` — unbounded FIFO accepting `write()` on one side, `get/peek` on the other. Scoreboard pulls data at its own pace.

```
class router_sb extends uvm_scoreboard;
 uvm_tlm_analysis_fifo #(yapp_packet) yapp_fifo = new("yapp_fifo", this);
 task run_phase(uvm_phase phase);
 yapp_packet pkt;
 forever begin
 yapp_fifo.get_peek_export.get(pkt); // blocking get
 // compare with expected...
 end
 endtask
endclass
// Testbench connect_phase:
yapp.agent.monitor.yapp_out.connect(scoreboard.yapp_fifo.analysis_export);
```

Multiple Analysis Exports (Broadcast)

```
// One monitor port connected to two separate consumers:
yapp_mon.send_yapp.connect(scoreboard.yapp_in);
yapp_mon.send_yapp.connect(coverage_collector.yapp_in);
// Both write() implementations are called on every write()
```

TLM (Transaction Level Modeling)

Concepts

- **Data flow**: producer → consumer
- **Control flow**: initiator requests, target responds
- **Put (push)**: producer is initiator → pushes data to consumer
- **Get (pull)**: consumer is initiator → pulls data from producer
- **Blocking** (tasks): can suspend until complete; `put()`, `get()`, `peek()`
- **Non-blocking** (functions): return 0/1 immediately; `try_put()`, `try_get()`, `can_put()`, `can_get()`

TLM Objects

- **port**: in initiator component, provides access to implementation
- **imp**: in target component, contains the implementation (requires all method variants)
- **export**: for hierarchical routing of imp connections up the hierarchy

Uni-directional Connector Types

`uvm_put_port/imp/export`, `uvm_get_port/imp/export`, `uvm_get_peek_port/imp/export`, `uvm_blocking_put_port/imp/export`, `uvm_nonblocking_put_port/imp/export`

Connection Rule

`connect()` is called off the **port** (initiator), with target (imp/export) as argument.

```
// Producer/Initiator declares port:
uvm_put_port #(yapp_packet) txn_out = new("txn_out", this);
txn_out.put(item); // blocking put
// Consumer/Target declares imp (must implement ALL methods for connector type):
uvm_put_imp #(yapp_packet, yapp_con) txn_in = new("txn_in", this);
task put(input yapp_packet pkt); /* process */ endtask
function bit try_put(input yapp_packet pkt); /* ... */ endfunction
function bit can_put(); return valid; endfunction
// Testbench connect_phase:
producer.txn_out.connect(consumer.txn_in);
```

TLM FIFO

`uvm_tlm_fifo #(T)` — buffered component between producer and consumer.

- `put_export` (put methods) and `get_peek_export` (get/peek methods)
- Constructor: `new(name, parent, size)` — default size=1; 0=unbounded
- **Both sides must be connected** in connect_phase

```
uvm_tlm_fifo #(yapp_packet) fifo = new("fifo", this, 5); // size 5
producer.send_txn.connect(fifo.put_export);
consumer.get_txn.connect(fifo.get_peek_export);
```

FIFO status methods: `size()`, `used()`, `is_empty()`, `is_full()`, `flush()`

Register Modeling (UVM RAL)

UVM Register Abstraction Layer Overview

- `uvm_reg_block` — top-level container; holds registers, memories, address maps
- `uvm_reg` — models one register; contains fields
- `uvm_reg_field` — models bit fields within a register (access policy, reset value)
- `uvm_mem` — models memory blocks
- `uvm_reg_map` — defines address mapping and access policy for one physical interface

If only one interface: named `default_map`. Multiple interfaces → multiple maps, same register can appear with different addresses/policies.

Register API Access Methods

```
// Front-door access (via UVC transactions — takes clock cycles):
yapp_regs.ctrl_reg.write(status, 8'h14);
yapp_regs.ctrl_reg.read(status, rdata);
// Back-door access (via hierarchical DUT path — zero time):
yapp_regs.ctrl_reg.read(status, rdata, UVM_BACKDOOR);
// With explicit map (multi-map):
model.reg1.write(status, 8'h14, UVM_FRONTDOOR, map1);
```

Integration in Testbench

```
class router_tb extends uvm_env;
 yapp_router_regs_t yapp_rm; // register model handle (generated type)
 hbus_reg_adapter reg2hbus; // adapter: converts register ops to UVC transactions
 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 yapp_rm = yapp_router_regs_t::type_id::create("yapp_rm", this);
 yapp_rm.build(); // must explicitly build data hierarchy
 yapp_rm.lock_model(); // creates address map, no further changes
 yapp_rm.default_map.set_auto_predict(1); // enable implicit prediction
 yapp_rm.set_hdl_path_root("hw_top.dut"); // root for backdoor paths
 reg2hbus = hbus_reg_adapter::type_id::create("reg2hbus", this);
 endfunction
 function void connect_phase(uvm_phase phase);
 yapp_rm.default_map.set_sequencer(hbus.masters[0].sequencer, reg2hbus);
 endfunction
endclass
```

Prediction Modes

- **Implicit Prediction** (`set_auto_predict(1)`): register model auto-updated for all API-driven DUT accesses. Easiest but only captures API accesses.
- **Explicit Prediction**: required when DUT registers also accessed by direct UVC sequences (not via API). A predictor component monitors UVC transactions and updates model.

Built-in Register Sequences

- `uvm_reg_hw_reset_seq` — read all registers, check reset values
- `uvm_reg_bit_bash_seq` — walk 0s/1s through each bit per access policy
- `uvm_reg_access_seq` / `uvm_mem_access_seq` — write frontdoor, read backdoor (and vice versa)
- `uvm_mem_walk_seq` — write walking pattern to memory, check readback
- `uvm_reg_mem_hdl_paths_seq` — verify backdoor paths accessible

```
// Executing a built-in sequence:
class uvm_reset_test extends base_test;
 uvm_reg_hw_reset_seq reset_seq;
 function void build_phase(uvm_phase phase);
 reset_seq = uvm_reg_hw_reset_seq::type_id::create("uvm_reset_seq");
 super.build_phase(phase);
 endfunction
 task run_phase(uvm_phase phase);
 phase.raise_objection(this);
 reset_seq.model = tb.yapp_rm; // point to testbench register model
 reset_seq.start(null); // null: sequencer pre-set via set_sequencer()
 phase.drop_objection(this);
 endtask
endclass
```

Model Generation from IP-XACT

Hand-coding register models for large designs is error-prone. Tools like Cadence `reg_verifier` generate UVM register model code from IP-XACT (IEEE 1685) XML specifications:

```
reg_verifier -domain uvmreg -top <IP-XACT.xml> -dut_name <name> [-cov] [-quicktest]
```

UVM Class Hierarchy

```
uvm_void
└── uvm_object                  (1-arg constructor: name)
    ├── uvm_sequence_item       ← data items, packets
    │   └── uvm_sequence        ← stimulus sequences
    ├── uvm_report_object
    │   └── uvm_component       (2-arg constructor: name, parent)
    │       ├── uvm_driver      ← drives DUT pins
    │       ├── uvm_monitor     ← observes DUT pins
    │       ├── uvm_sequencer   ← manages sequence-driver flow
    │       ├── uvm_agent       ← contains sequencer + driver + monitor
    │       ├── uvm_env         ← contains agents and other UVCs
    │       ├── uvm_test        ← top-level test class
    │       └── uvm_scoreboard  ← checks DUT output
    └── uvm_transaction         (legacy; prefer uvm_sequence_item)
```

UVM Architecture Summary

Complete Testbench Hierarchy

```
module tb_top
└── Test (uvm_test)
    └── Testbench (uvm_env)
        ├── YAPP UVC (uvm_env)
        │   └── Agent (uvm_agent)
        │       ├── Sequencer ←── sequence items ←── Sequences
        │       ├── Driver ──────── virtual interface ──▶ DUT
        │       └── Monitor ─── analysis_port ──▶
        ├── HBUS UVC / Channel UVCs
        ├── Multichannel Sequencer (coordinates multiple UVCs)
        ├── Scoreboard ◀── analysis connections from monitors
        └── Register Model ◀── adapter from HBUS UVC
```

Data Flow Summary

1. Test configures environment via `uvm_config_db`, selects sequences
1. Sequences use `uvm_do` macros to randomize and send data items
1. Sequencer arbitrates item requests from driver
1. Driver receives items via TLM get_next_item, drives DUT pins via virtual interface
1. Monitor passively samples DUT signals, builds packet, broadcasts via analysis port
1. Scoreboard receives packet, compares with expected from reference model
1. Coverage collector samples functional coverage
1. Register model maintains shadow copies of DUT registers via adapter

FIFO

Synchronous FIFO

The `w_ptr` and `r_ptr` and circulating with lower 3 bits the actual address inside the FIFO. 4th bit is wrapper bit.

```
module syn_fifo #(parameter DEPTH = 8, WIDTH = 8)(
 input clk,
 input rstn,
 input [WIDTH-1:0] w_data,
 output reg full,
 input w_en,
 output reg [WIDTH-1:0] r_data,
 output reg empty,
 input r_en
);
 parameter ptr_depth = $clog2(DEPTH); // 3
 reg [ptr_depth:0] w_ptr, r_ptr; // 1+logN bits = 4bits
 reg [WIDTH-1:0] fifo [DEPTH-1:0];
 always_ff @(posedge clk or negedge rstn ) begin : write_side
 if (!rstn) begin
 w_ptr <= 0;
 end else (w_en && !full) begin
 w_ptr <= w_ptr + 1;
 fifo[w_ptr[0 +: ptr_depth]] <= w_data;
 end
 end
 always_ff @(posedge clk or negedge rstn ) begin : read_side
 if (!rstn) begin
 r_ptr <= 0;
 r_data <= 0;
 end else (r_en && !empty) begin
 r_ptr <= r_ptr + 1;
 r_data <= fifo[r_ptr[0 +: ptr_depth]];
 end
 end
 assign empty = (w_ptr == r_ptr);
 assign full = ((w_ptr[ptr_depth] != r_ptr[ptr_depth]) && (w_ptr[0 +
: ptr_depth] == r_ptr[0 +: ptr_depth]));
endmodule
```

Asynchronous FIFO

```
module async_fifo #(
 parameter int DEPTH = 8,
 parameter int WIDTH = 8,
 localparam int ADDR_W = $clog2(DEPTH)
) (
 input logic clk_wr,
 input logic clk_rd,
 input logic rstn,
 input logic [WIDTH-1:0] w_data,
 input logic w_en,
 output logic full,
 output logic [WIDTH-1:0] r_data,
 input logic r_en,
 output logic empty
);
 logic [WIDTH-1:0] mem [0:DEPTH-1];
 logic [ADDR_W:0] wptr_bin, wptr_bin_n;
 logic [ADDR_W:0] wptr_gray, wptr_gray_n;
 logic [ADDR_W:0] rptr_bin, rptr_bin_n;
 logic [ADDR_W:0] rptr_gray, rptr_gray_n;
 logic [ADDR_W:0] rptr_gray_wr_sync1, rptr_gray_wr_sync2;
 logic [ADDR_W:0] wptr_gray_rd_sync1, wptr_gray_rd_sync2;
 logic [ADDR_W:0] wq2_rptr_gray_inv2msb;
 function automatic logic [ADDR_W:0] bin2gray(input logic [ADDR_W:
0] b);
 return (b >> 1) ^ b;
 endfunction
 always_comb begin
 wptr_bin_n = wptr_bin;
 if (w_en && !full)
 wptr_bin_n = wptr_bin + 1'b1;
 wptr_gray_n = bin2gray(wptr_bin_n);
 end
 always_comb begin
 rptr_bin_n = rptr_bin;
 if (r_en && !empty)
 rptr_bin_n = rptr_bin + 1'b1;
 rptr_gray_n = bin2gray(rptr_bin_n);
 end
 always_ff @(posedge clk_wr or negedge rstn) begin
 if (!rstn) begin
 wptr_bin <= '0;
```

Cache

Avoid Cache Miss

1. **Compulsory**

   First fetch will always miss since cache only store previsited data. **Prefetching** can lower miss rate.

1. **Capacity**

   Increate Cache capacity to lower miss rate.

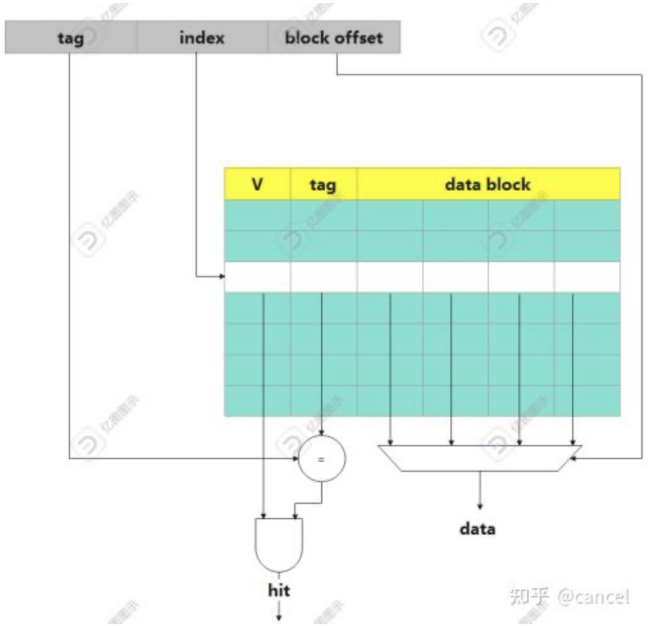
1. **Conflict**

   Data

Types of Caches

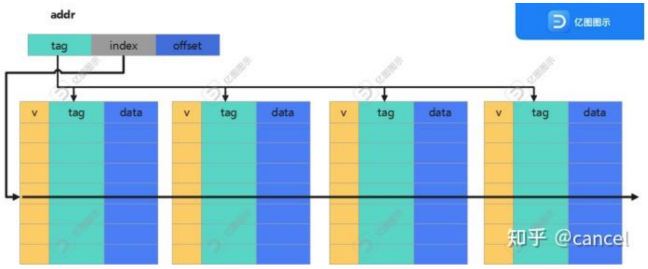
Direct Mapped

Each `index` maps to exactly one cache line. Simple and fast, but prone to conflict misses when multiple addresses map to the same cache line.



Set Associative

1. Use `index` to locate cacheline
1. Use `tag` to locate the block(set)
1. `Offset` to identify the byte



$$\text{Offset bits} = \log_2(\text{Block Size})$$

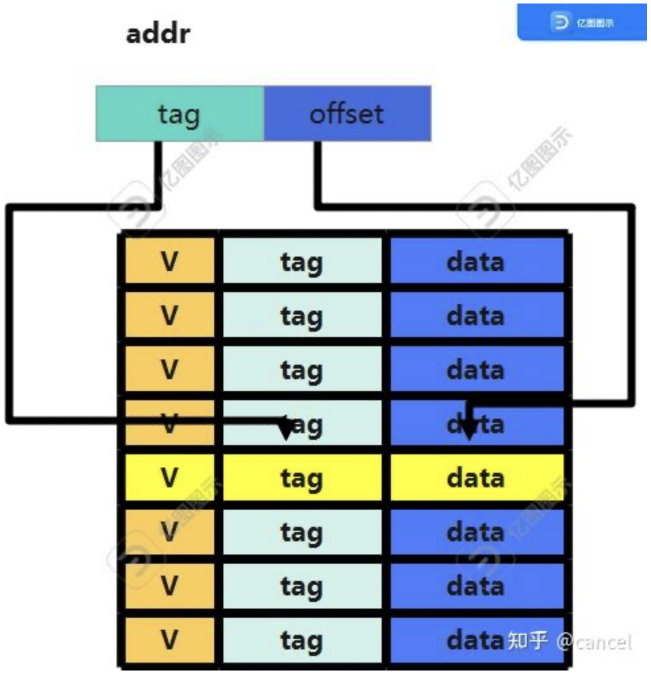
$$\text{Index bits} = \log_2\left(\frac{\text{Cache Data Size}}{\text{Block Size} \times \text{Ways}}\right)$$

$$\text{Tag bits} = \text{Address Width} - \text{Index bits} - \text{Offse}$$

Fully Associative

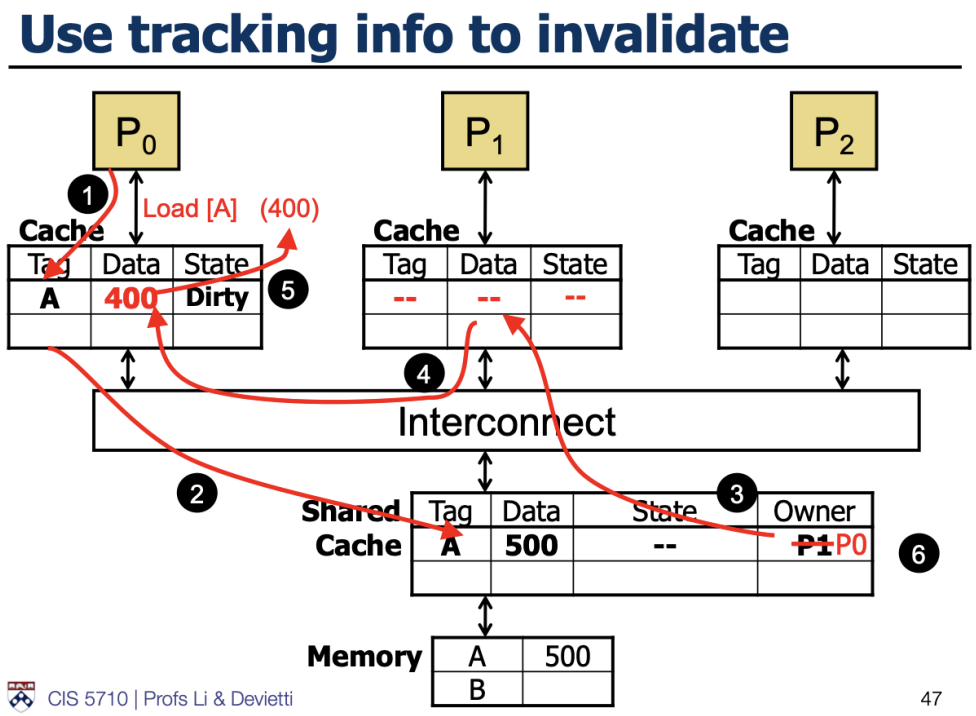
Only compares tag and valid. Every cacheline only has one tag.

Requires more complex hardware (comparators for all entries) and slower access time.



Cache Coherence

VI(Valid Invalid)



MESI

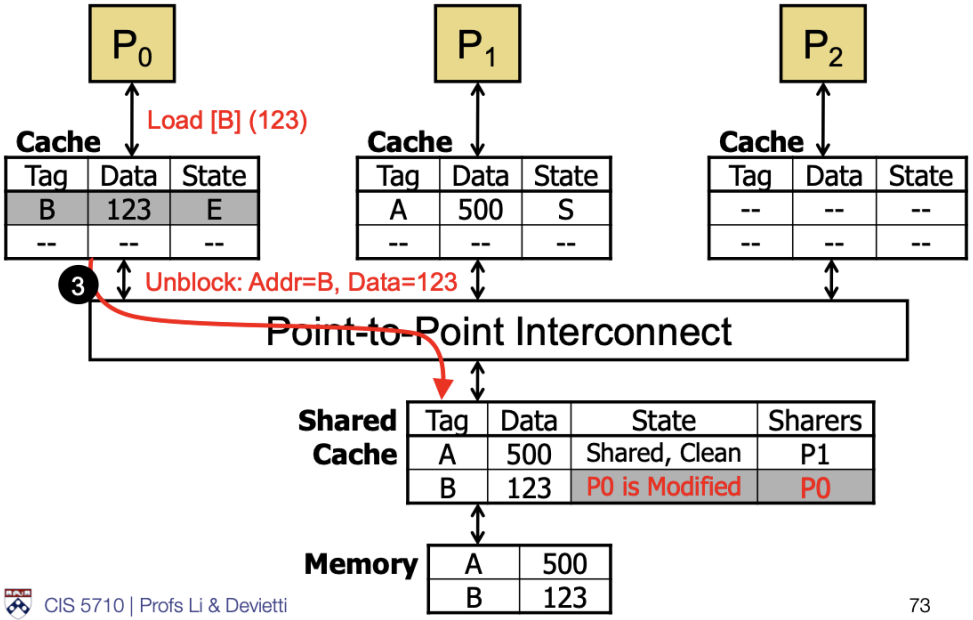
- Exclusive has read/write permissions just like Modified but clean instead of dirty.
- Eliminates the cost of coherence when there's no sharing.

MESI State Transition Table

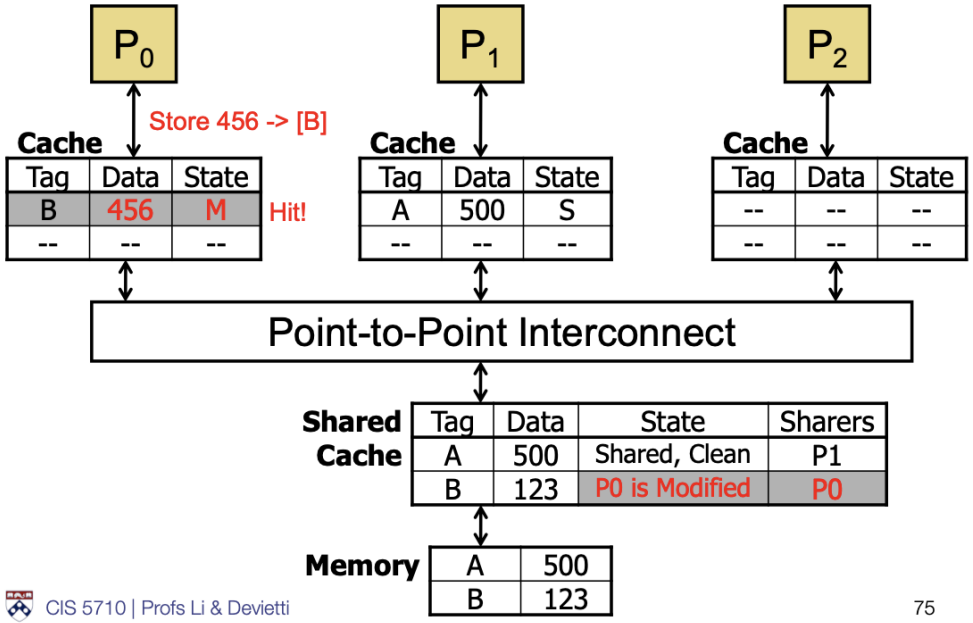
State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss ➔ **S or E**	Miss ➔ M	---	---
Shared (S)	Hit	Upg Miss ➔ M	Send Data ➔ S	Send Data ➔ I
Exclusive (E)	Hit	**Hit ➔ M**	Send Data ➔ S	Send Data ➔ I
Modified (M)	Hit	Hit	Send Data ➔ S	Send Data ➔ I

* Load Miss goes to E if no other processor hold this line

MESI Coherence Example: Step #5



MESI Coherence Example: Step #7



MSI(Modified Shared Invalid)

- Modified data are local DIRTY copy.
- Shared data are local CLEAN copy & READ-ONLY.

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Load Miss ➔ S	Store Miss ➔ M	---	---
Shared (S)	Hit	Upgrade Miss ➔ M	Send Data ➔ S	Send Data ➔ I
Modified (M)	Hit	Hit	Send Data ➔ S	Send Data ➔ I

MOESI

- Shared block is read-only and clean
- **Owned block is read-only and dirty**

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss ➔ S or E	Miss ➔ M	---	---
Shared (S)	Hit	Upg Miss ➔ M	Send Data ➔ S	Send Data ➔ I
Owned (O)	Hit	Upg Miss ➔ M	Send Data ➔ O	Send Data ➔ I
Exclusive (E)	Hit	Hit ➔ M	Send Data ➔ S	Send Data ➔ I
Modified (M)	Hit	Hit	Send Data ➔ O	Send Data ➔ I

* Load Miss goes to E if no other processor hold this line.

Register Renaming

Why Renaming? — False (Name) Dependencies

In a pipelined or out-of-order processor, **data hazards** between instructions can stall execution. There are three types:

Hazard	Name	Example	False?	RAW	Read After Write	`ADD x1, x2, x3` then `SUB x4, x1, x5`	True depende — cannot elimi
WAR	Write After Read	`ADD x4, x1, x2` then `SUB x1, x3, x5`	False — x1 in SUB is a different logical value	WAW	Write After Write	`ADD x1, x2, x3` then `SUB x1, x4, x5`	False — both write x1 but to independent values

> **WAR and WAW are name (false) dependencies** — they only exist because two unrelated instructions use the same *architectural register name*. Register renaming eliminates them entirely by giving each write its own fresh physical register.

Key Hardware Structures

Structure	Role	**Rename Map Table (RMT)** / Register Alias Table (RAT)	Maps each architectural register to its current physical register. Updated every rename cycle.
Free List	Pool of unallocated physical registers. One is popped per destination register on rename; one is pushed back per commit (the stale register).	**Reorder Buffer (ROB)**	Tracks all in-flight instructions in program order for in-order commit. Stores (dest arch reg, new phys reg, stale phys reg) per instruction.

Rename Step-by-Step (per instruction)

1. **Read sources** — look up each source arch reg in the RMT → get physical source registers
1. **Allocate destination** — pop a free physical register from the Free List
1. **Record the stale register** — the physical reg previously mapped to this arch dest (freed at commit)
1. **Update the RMT** — write the new arch → physical mapping
1. **Enqueue into ROB** — log source physicals, dest physical, and stale physical

Worked Example

Assume: RMT = `{x1→p1, x2→p2}`, Free List = `[p3, p4, p5, ...]`

```
// Instruction 1
ADD x1, x1, x1
  sources:  x1→p1, x1→p1
  allocate: p3 from free list
  renamed:  ADD p3, p1, p1
  stale:    p1  (old x1 mapping — freed at commit)
  RMT now:  {x1→p3, x2→p2}

// Instruction 2
SUB x2, x1, x2
  sources:  x1→p3, x2→p2   (reads updated RMT)
  allocate: p4 from free list
  renamed:  SUB p4, p3, p2
  stale:    p2
  RMT now:  {x1→p3, x2→p4}

// Instruction 3 — WAW hazard with Instr 1 is ELIMINATED
ADD x1, x2, x2
  sources:  x2→p4
  allocate: p5 from free list
  renamed:  ADD p5, p4, p4
  stale:    p3  (no dependency on Instr 1!)
  RMT now:  {x1→p5, x2→p4}
```

Instr 3 can execute **independently** of Instr 1 even though both write `x1`, because they now target distinct physical registers `p3` and `p5`.

Commit and Free List Reclamation

Physical registers are freed **only at commit**, not at execution — this is critical for supporting rollback:

- Instr 1 commits → **p1** (stale) returned to Free List
- Instr 2 commits → **p2** (stale) returned
- Instr 3 commits → **p3** (stale) returned

> Never free on execution — the stale register must survive in case the instruction is squashed before commit.

Rollback on Misprediction / Exception

All instructions younger than the faulting instruction are squashed:

1. **Restore the RMT** — from a checkpoint saved at the branch (or by walking the ROB backwards)
1. **Return physical registers** — all physicals allocated to squashed instructions go back to the Free List
1. **Flush the ROB** — remove all entries younger than the faulting instruction

The visible architectural state after rollback must exactly match the state at the point of the branch/exception.

Corner Cases — DV Focus

Corner Case	Expected Behavior	**Free List empty**	Rename stage stalls; no instruction may enter rename until commit frees a stale register
ROB full	Fetch/decode stalls; structural hazard — must wait for ROB head to commit	**Zero register (x0)**	Writes to x0 are discarded — x0 always reads as 0, never renamed or allocated a physical register
Back-to-back writes to same arch reg	Each write gets a new physical reg; downstream readers must pick up the *latest* RMT mapping	**Superscalar rename (multiple insns/cycle)**	If two insns in the same rename group write the same arch reg, the later one wins; forwarding within the group is required
Squash + commit racing	Mispredicting instruction commits its own stale register and simultaneously squashes all younger insns atomically	**Double-free of stale register**	Stale regs of squashed insns return to Free List at squash time; must NOT be freed again at would-be commit

Verification Strategy

- **Golden reference model** — software shadow of RMT and Free List stepping in lockstep with the DUT; compare after every rename cycle
- **Free List count invariant** — `|phys_regs| = |active RMT mappings| + |free list| + |stale pending commit|` must hold at all times
- **Rollback self-check** — after every squash, verify RMT matches the pre-branch checkpoint; verify Free List count increases by the correct amount
- **x0 property** — assert x0's physical mapping never changes; x0 reads always return 0
- **Stall coverage** — cover Free List empty and ROB full; verify all upstream stages stall with no side effects

Dependency types

1. **RAW**(Read After Write)
1. **WAW**(Write After Write)
1. **WAR**(Write After Read)

Arbiter

Fixed Priority Arbiter

Solution 1

Find last 1, a number & it's 2's complement number result in the last 1 position.

```
assign gnt = req & (~(req - 'd1));
```

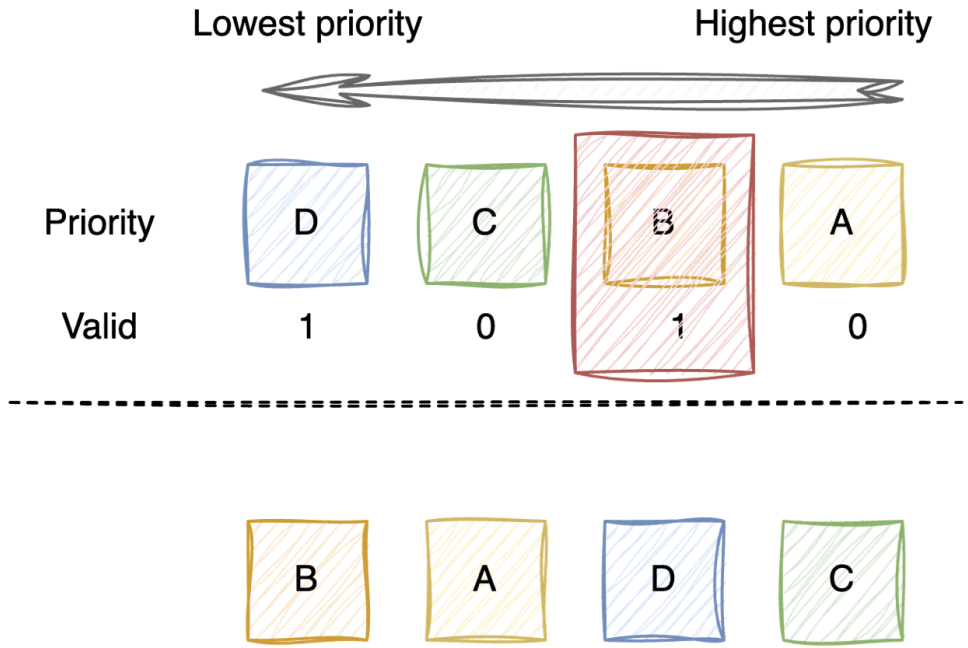
Solution 2

```
reg [LENGTH-1:0] pre_req;
 pre_req[0] = 1'b0;
 pre_req[LENGTH-1:1] = req[LENGTH-2:0] | pre_req[LENGTH-2:0];
 gnt = req & ~pre_req;
```

Round Robin Arbiter

Principle

After a grant, the requester on its left has highest priority.



RTL Realisation

```
module #(parameter N = 8) rr_abt(
 input clk, rstn,
 input logic [N-1:0] req,
 output logic [N-1:0] gnt
);
 // unmasked simple arbiter
 logic [N-1:0] unmasked_higher_pri_req;
 logic [N-1:0] unmasked_gnt;
 assign unmasked_higher_pri_req[0] = 0;
 assign unmasked_higher_pri_req[N-1:1] =
unmasked_higher_pri_req[N-2:0] | req[N-2:0];
 assign unmasked_gnt = req & ~unmasked_higher_pri_req;
 // masked simple arbiter
 logic [N-1:0] masked_higher_pri_req;
 logic [N-1:0] masked_gnt;
 logic [N-1:0] masked_req;
 assign masked_req = req & ptr;
 assign masked_higher_pri_req[0] = 0;
 assign masked_higher_pri_req[N-1:1] = masked_higher_pri_req[N-2:0]
| masked_req[N-2:0];
 assign masked_gnt = masked_req & ~masked_higher_pri_req;
 logic [N-1:0] ptr;
 always_ff @( posedge clk, negedge rstn ) begin
 if (!rstn) ptr <= {N{1'b1}};
 else begin
 if(|masked_req) ptr <= masked_higher_pri_req;
 else if (|req) ptr <= unmasked_higher_pri_req;
 end
 end
 assign gnt = (|masked_req) ? masked_gnt : unmasked_gnt;
endmodule
```

Formal Verification

Assertion

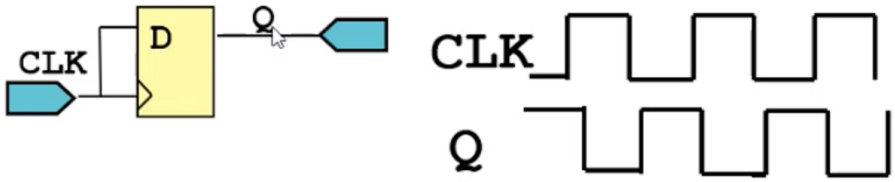
Clock behavior

This is always false due to clk2Q delay.

```
FAL_AST: assert property (@(posedge clk) clk);
```

Next cycle implication with 2 clocks. The second assert will happen at the next posedge CLK2,

```
NEXT_CYCLE: assert property (@(posedge CLK1) EN1 |=> @(posedge
CLK2) EN2);
```



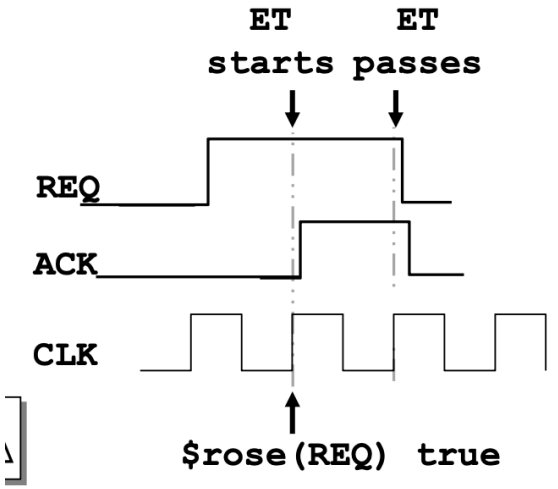
Assertion Conditions

`$rose()` and `$fall()`

To resolve assertion overlapping

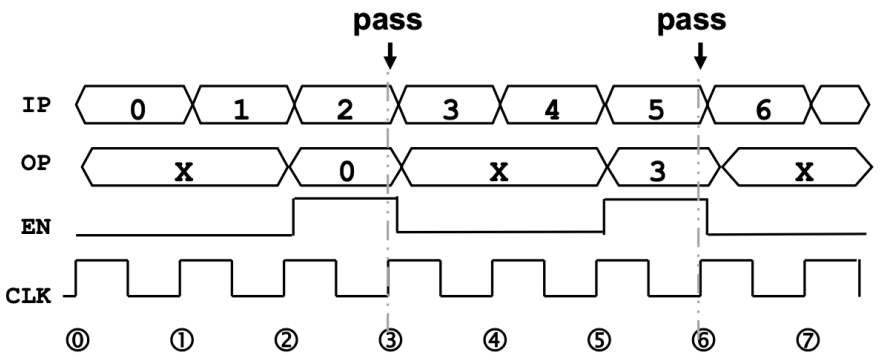
```
property sovled;
 @(posedge CLK) $rose(REQ) |=> ACK;
endproperty: overlapping_solved
assert property solved;
```

> Note: `$rose(A)` is equivalent to `(A != 1'b1 ##1 A)` since it is also true for `'X → 1` or `` `Z -> 1 ``.



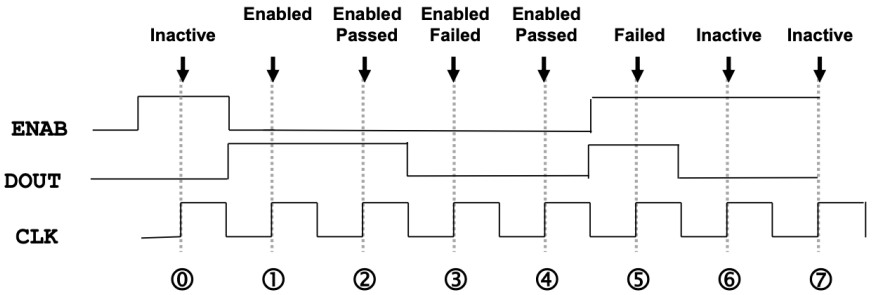
`$past(A, N)`, N is default to 1.

```
property PAST;
 @(posedge CLK) EN -> (OP == $past(IP, 2));
endproperty: past_N
```



`$stable()` and `$changed()`

```
property STABLE1;
 @(posedge CLK) ~ENAB |=> $stable(DOUT);
endproperty
Equivalent to:
property SAME;
 @(negedge CLK) ( ~ENAB |=> (DOUT == $past(DOUT)) );
endproperty
```



`$countones()` and `$isunknown()`

- `$countones()`
  - Returns an integer representing the number of 1's in expr1
  - Any value other than 1 ignored
- `$isunknown()`
  - Returns a boolean `true` if any bit in expr1 is `x` or `z`

Disable and Synchronous Abort

```
default disable iff(RST)
```

`disable iff` is asynchronous.

For synchronous abortion,

`sync_accept_on`

- If we observe `A ##1 B && CLR ##1 !C`, the property passes on the cycle where `CLR` occurred.

```
ABORT_PASS_ON_CLR: assert property @(posedge CLK) A |=> sync_accept_on(CLR) (B ##1 C ##1 D);
```

`sync_reject_on`

- If we observe `A ##1 B ##1 C && CLR ##1 D`, the property fails on the cycle where `CLR` occurred.

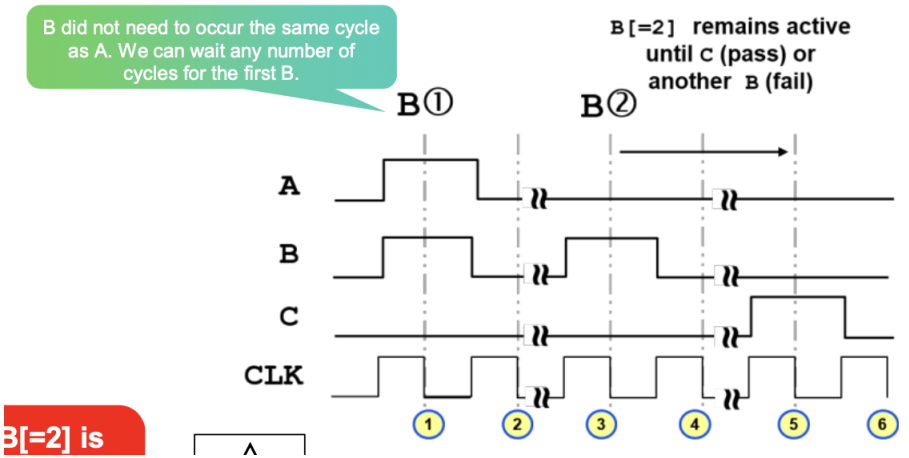
```
ABORT_FAIL_ON_CLR: assert property @(posedge CLK) A |=> sync_reject_on(CLR) (B ##1 C ##1 D);
```

Repetition

Non-consecutive Repetition

```
property NONCONSEC_REP;
 @(negedge CLK)
 A |-> B[=2] ##1 C;
endproperty
```

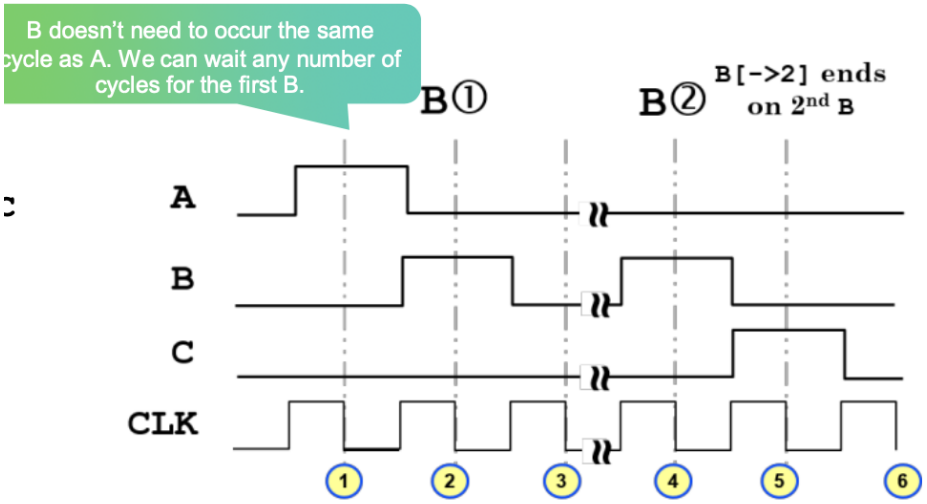
> **No restriction on number of cycles between the first and second B**
>
> **No restriction on number of cycles between second B and C**



Go-to Repetition

```
property GOTO_REP;
 @(negedge CLK)
 A |-> B[->2] ##1 C;
endproperty
```

> **No restriction on number of cycles between the first and second B**
>
> **Last occurrence of B must be followed immediately by C**



Coverage

Cover Sequence

Cannot use implication in `cover sequence(expr)`.

Example

✓
```
cover sequence ( A ##1 B);
```

✗
```
cover property ( A |=> B);
```

This means either:

1. A followed by B
1. A not occurring (remembering what an implication means)

✗
```
cover sequence ( A |=> B);
```

Illegal syntax

Liveness Properties

Weak Properties

Do not require `expr` to be executed.

Directives default to weak:

1. `assume`
1. `assert`
   `eventually`

   Must have a bounded range, cannot use `$` as upper bound.

   ```
   assert property(
    REQ |-> eventually[1:3] GNT);
   ```

   This is equivalent to:

   ```
   assert property(
    REQ |-> weak(##[1:3] GNT));
   ```

Strong Properties

Must detect the `expr` for the formal verification to `$finish`.

Directives default to strong:

1. `cover`
1. `restrict`
   `s_eventually`

   May not have a range.

   ```
   assert property(
    REQ |-> s_eventually[1:3] GNT);
   ```

   This is equivalent to:

   ```
   assert property(
    REQ |-> strong(##[1:3] GNT));
   ```

Property Reuse

Sequence Reuse

Parameterized Sequence

```
sequence N_CYCLE_RESP(n, p);
 (!p[*1:n-1] ##1 p);
endsequence
```

Instantiation

```
( $rose(req) ##1
 N_CYCLE_RESP(10,ack) ##1 done )
```

Result after expansion

```
( $rose(req) ##1 !ack[*1:9] ##1 ack ##1 done )
```

Property Reuse

Parameterized Sequence

```
property ResultAfterN(
 START, N, RESULT, STOP);
 @(posedge CLK) disable iff (STOP)
 START |=> ##N RESULT;
endproperty
```

Instantiation

```
A1 : assert property (
 ResultAfterN(WRITE_REQ,
 3, ACK,
 CANCEL));
```

Result after expansion

```
A1 : assert property (
 @(posedge CLK) disable iff (CANCEL)
 WRITE_REQ |=> ##3 ACK);
```

Generating Assertions

Generate 8 individual properties, the name `genvres` must be given.

```
reg [7:0] REQ, ACK;
property VRES (N);
 @(posedge CLK)
 $rose(REQ[N]) |=> ACK[N];
endproperty
genvar i;
generate
for (i=0; i<8; i=i+1)
 begin :genvres
 A1 : assert property (VRES(i));
 end
endgenerate
```

Instantiation

```
….genvres[0].A1
….genvres[1].A1
…
….genvres[7].A1
// Example Usage
// assert property topModule.U1.genvres[0].A1;
```

Assertion binding

Instantiates a verification component into any design module in the hierarchy.

Example

Verification component:

```
module mone_prop(input a,b,clk);
 property P1;
 @(posedge clk) $rose(a) |=> b;
 endproperty
 A1 : assert property (P1);
endmodule
```

Design Module:

```
module mone(input logic clk, req,
 output logic ack,
 ...);
...
endmodule
```

Testbench, where binding is used:

```
module top;
...
 // instantiate design modules
 mone m1 (clk, req1, ack1...);
 mone m2 (clk, req2, ack2...);
 bind mone mone_prop mp1(req,ack,clk);
```

If 1 specific mone instance need binding: `bind mone:m1 mone_prop mp1(req,ack,clk);`

Auxiliary Code

Case study

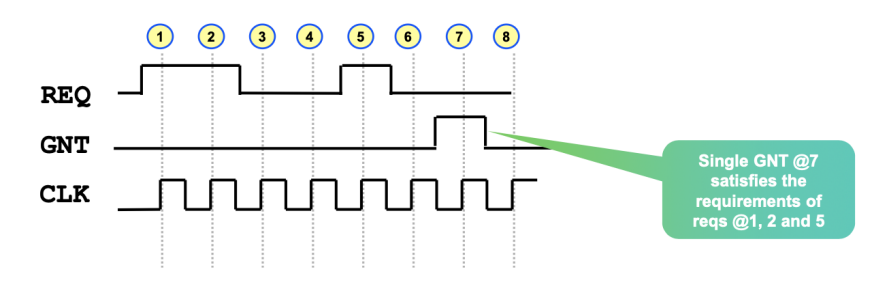
Required request/grant interface behavior:

1. If you have a REQ, then you must have a GNT within 2 to 8 cycles
1. Requests can overlap

For example, you can have another request before the first grant received.

1. Each REQ must have a corresponding GNT
1. No GNT without a REQ

```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```



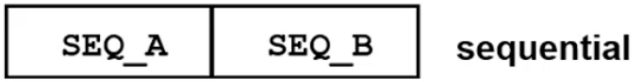
The overlapping nature of REQ and GNT is making this incorrect.
Single assertion will not be possible

```
for (i=7; i >= 2; i--) //only operate on MAIN_SHIFT_REG elements
 if (GNT && (SHIFT_REG[i] == 1'b1)) //remove oldest request
 SHIFT_REG[i] <= 1'b0; //only remove a maximum of one request from shift reg per clock
 break;
end
```

Sequence Operations

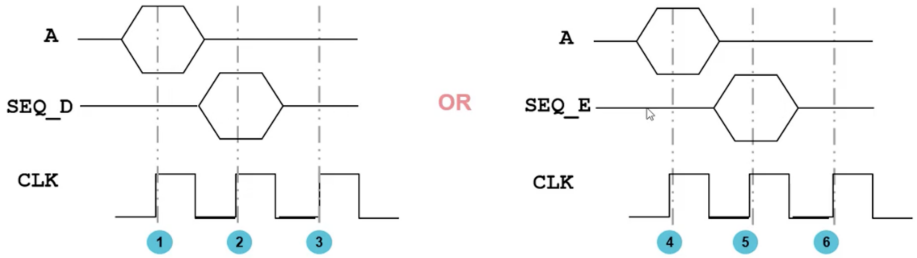
Operators with other sequences

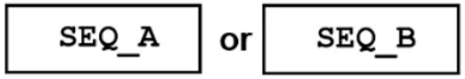
`##` : Cycle delay allow sequential or 2 sequences overlap by 1 cycle



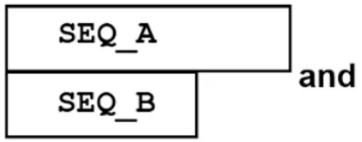


`or` : Either SEQ_A or SEQ_B is true, **or both**





`and` : Start at same cycle, end can be different

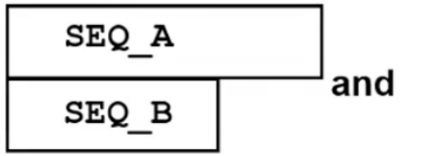


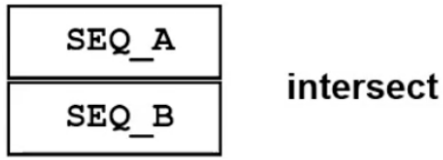
`intersect` : Must start and end at the same cycle, length matching

Example

`EN1`, `DPULSE` must occur between 4 and 8 times over the next 15 cycles.

```
sequence S1;
 EN1 |=> (DPULSE[=4:8]) intersect (1'b1[*15]);
endsequence
```





Other operations

`(expr) throughout (seq)`

Equivalent to:

```
(expr)[*0:$] intersect (seq)
```

`(expr) within (seq)`

Advanced SVA

Endpoints

`seq.triggered`: A signal being true when `seq` is true, last for 1 clock cycle defined.

This can be used when describing multiple concurrent or open-ended sequences.

→ P173

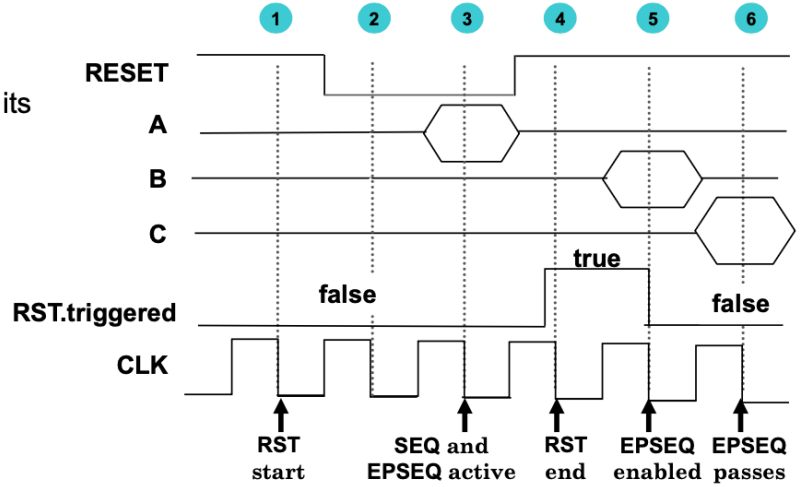
Using Sequence Endpoints

An endpoint can be used in a sequence

An endpoint is **not** equivalent to the sequence of its definition

In `EPSEQ`, `RESET` will be looked for before `A`

In `SEQ`, the `RESET` will only be checked after `A`




```
sequence RST;
   (RESET ##1 !RESET[*2] ##1 RESET);
endsequence

property EPSEQ;
   @(negedge CLK) ((A ##1 RST.triggered ##1 B) |=> (C));
endproperty

property SEQ;
   @(negedge CLK) ((A ##1 RST ##1 B) |=> (C));
endproperty
```

? Question

When does the `SEQ` enabling sequence become incomplete?

173 © Cadence Design Systems, Inc. All rights reserved.

cadence

In property `SEQ`, the `RST` sequence must begin after `A` and be followed by `B`.

In property `EPSEQ`, you expect the end of the `RST` sequence to follow `A`. Therefore, the `RST` sequence must run in parallel with `A`. In fact, because the `RST` sequence is of fixed length, you can fully expand `EPSEQ` as follows:

```
(RESET ##1 ~RESET ##1 ~RESET && A ##1 RESET ##1 B)
```

Endpoint can be useful when describing multiple concurrent or open-ended sequences. Separate sequences can be individually defined and named, and a property declared to check endpoints in the correct order.

Example

Expland `WHATSEQ`:

```
sequence ONE; (A ##1 B[*3] ##1 C); endsequence
sequence TWO; (K ##1 L ##1 M ##1 N); endsequence
sequence THREE; (X ##2 Y[*2]); endsequence
sequence WHATSEQ;
 (ONE.triggered ##1 TWO.triggered ##1 THREE.triggered);
endsequence
```

Correct Answer:

```
sequence EXPANDED;
 A ##1 B ##1 (B & K) ##1 (B & L & X) ##1 (C & M) ##1 (N & Y) ##1 Y;
endsequence
```

Action Blocks

- Not synthesizable
- Includes both pass and fail statements

```
property HS;
 @(posedge CLK)
 REQ ##1 ~ACK |-> BUSY;
endproperty
A1 : assert property (HS)
 // pass block
 $display("A1 passed");
 else
 // fail block
 begin
 $warning("A1 failed");
 err_count++;
 -> ack_err_event;
 end
```

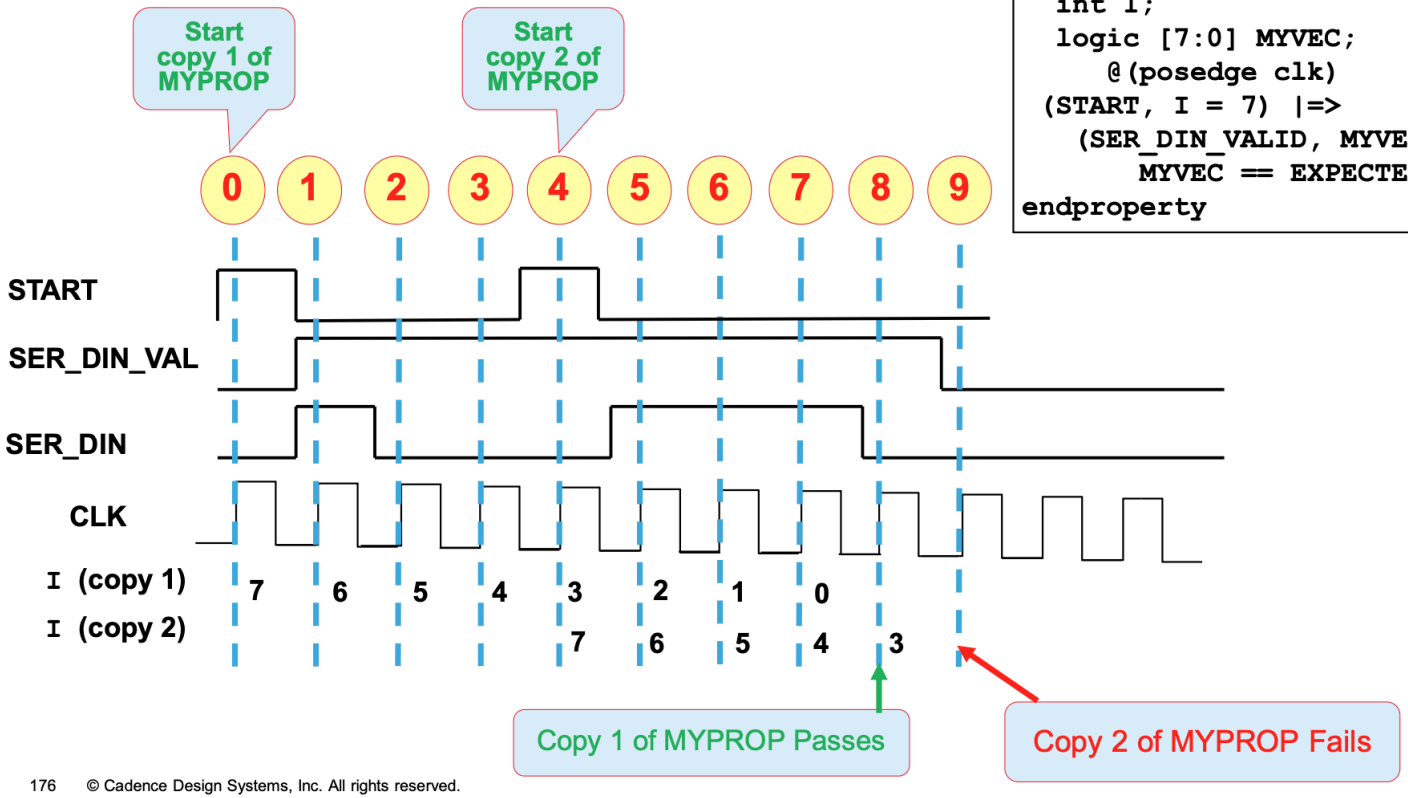
Local Variables to Deserialise

If we are checking a serial data rather than just a 1-bit value, we can check the serial data by sampling and assigning it to a local variable within property

```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
 int I; // Loop variable
 logic [7:0] MYVEC; // Local variable to store SER_DIN
 @(posedge clk)
 ( START, I = 7 ) |=> (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8] ##1 MYVEC == EXPECTED_VAL;
 // Start loop         Sampling SER_DIN                Wait for 8 bits    Check result
endproperty
```

What if we have a START signal within 8 cycles? Overlapping Properties will have different copies and will not interfere each other.

Copy1 and Copy 2 cannot communicate their own local value of I




```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
  int I;
  logic [7:0] MYVEC;
     @(posedge clk)
 (START, I = 7) |=>
   (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8] ##1
      MYVEC == EXPECTED_VAL;
endproperty
```

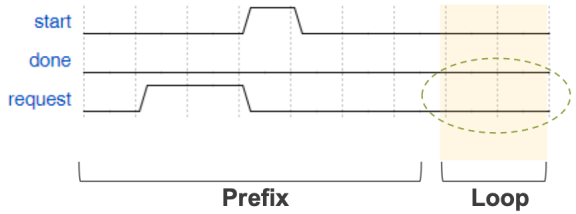
176 © Cadence Design Systems, Inc. All rights reserved.

Property Constructs

Livelocks

To resolve livelocks, define a property witch requires <expr> to be true.

```
assume property (
 s_eventually always !request
);
```



Followed-by Operator

After sequence is triggered, on same/next cycle, check if the properties are written correctly.

Overlapping:

```
<sequence> #-# <property>
```

Equivalent to:

```
not(sequence |-> not property)
```

Non-Ov

```
<seq
```

Equival

```
not(
```

`iff` Operator

Usage: `prop1 iff prop2`

nexttime[index]

Usage: `nexttime[index]<property expr 1>;` (`nexttime` and `nexttime[1]` are equivalent)

Example

```
D_NEXTT_E_2_F_W: assert property(D |-> nexttime[2] E ##1 F);
```

If D occurs then after 2 cycles, E is required to be true and F must be true in the cycle after that.

This is equivalent to

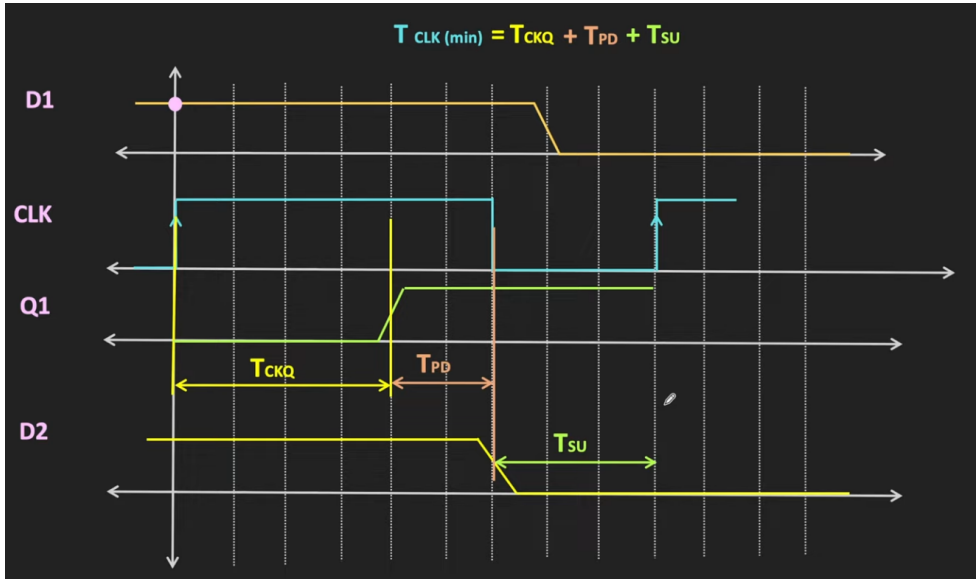
```
D_NEXTT_E_2_F_W: assert property(D |-> ##2 E ##1 F);
```

Clock Domain Crossing

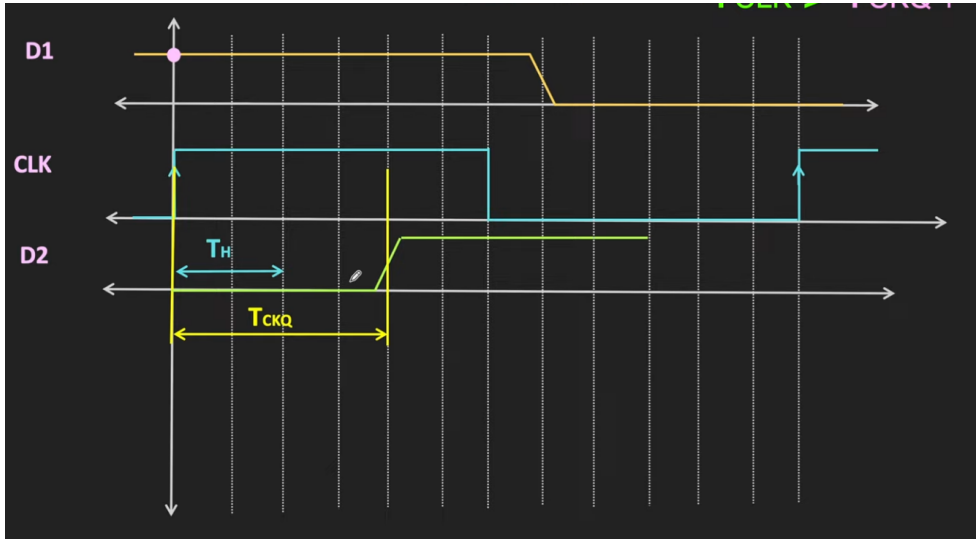
Timing violation

Two requirements for a digital circuit

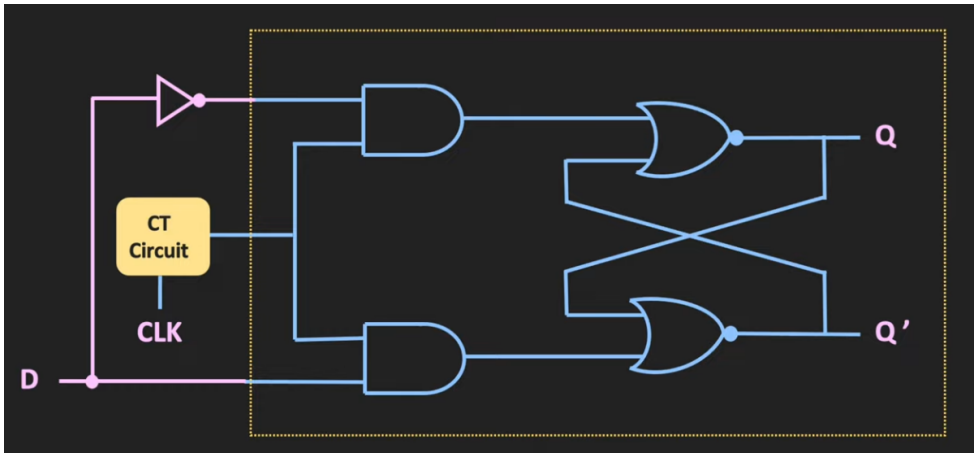
$$T_{CLK} > T_{pcq} + T_{pd} + T_{setup}$$



$$T_{pcq} > T_{hold}$$

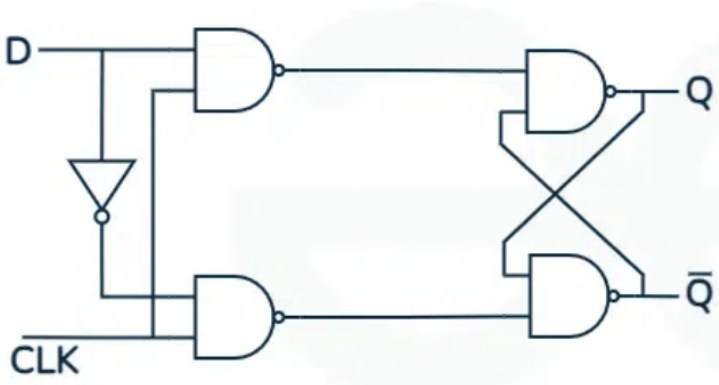


For a D flip flop, it has T_{plh} and T_{phl} (low to high and high to low propagation delay) due to time required to drive the logic gates in its circuit.



Types of Flip Flop

1. **D Flip Flop**



Truth Table

Q	D	$Q_{(t+1)}$
0	0	0
0	1	1
1	0	0
1	1	1

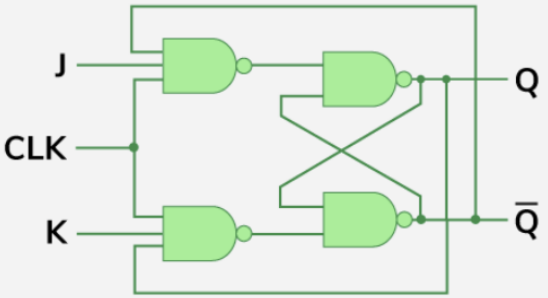
1. **JK Flip Flop**

   J=0, K=0 → hold

   J=0, K=1 → reset

J=1, K=0 → set

J=1, K=1 → toggle



Truth Table

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

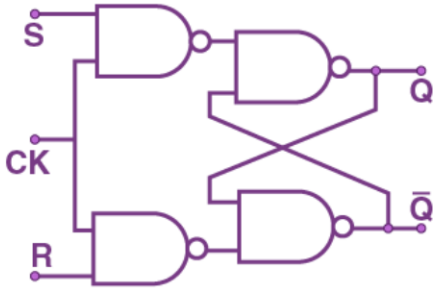
1. **SR Flip Flop**

   S=1, R=0 → set

   S=0, R=1 → reset

   S=0, R=0 → hold

   S=1, R=1 → invalid/forbidden in basic form



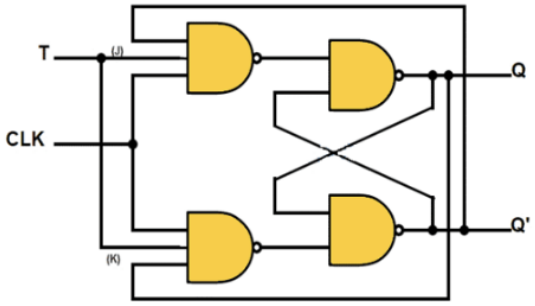
Truth Table

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

1. **T Flip Flop**

   T = 0 → Hold Output

   T = 1 → Toggle Output



T	Q	Q'
0	0	0
1	0	1
0	1	0
1	1	0

Sync and Async Reset

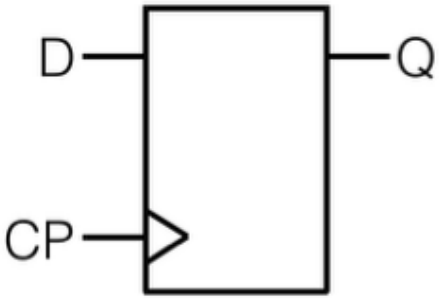
https://zhuanlan.zhihu.com/p/528860992

Comparison

1. Timing

   Sync reset added a logic gate, which is worth in timing compared to Async reset

   Sync



1. Area

   Due to added gate, async can have 10% less area.

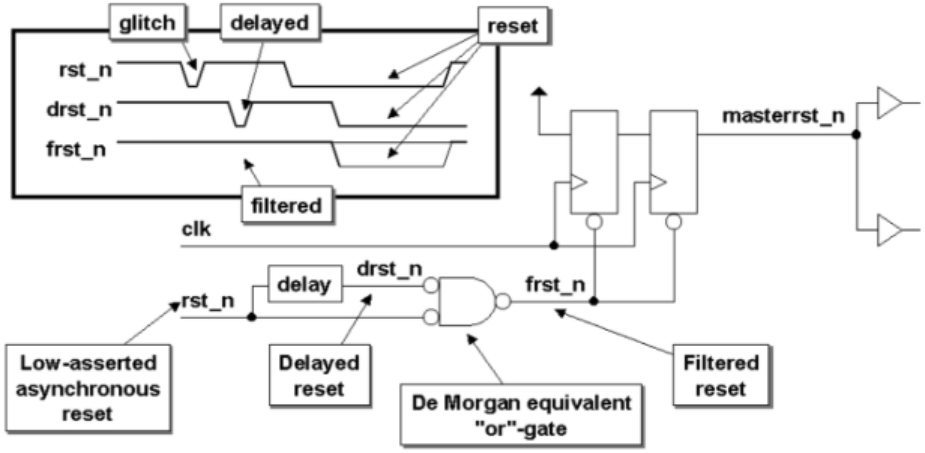
RTL realisation

Async reset requires a sufficiently long reset signal, otherwise a glitch will occur.

We can add a delay cell to filter out the reset.

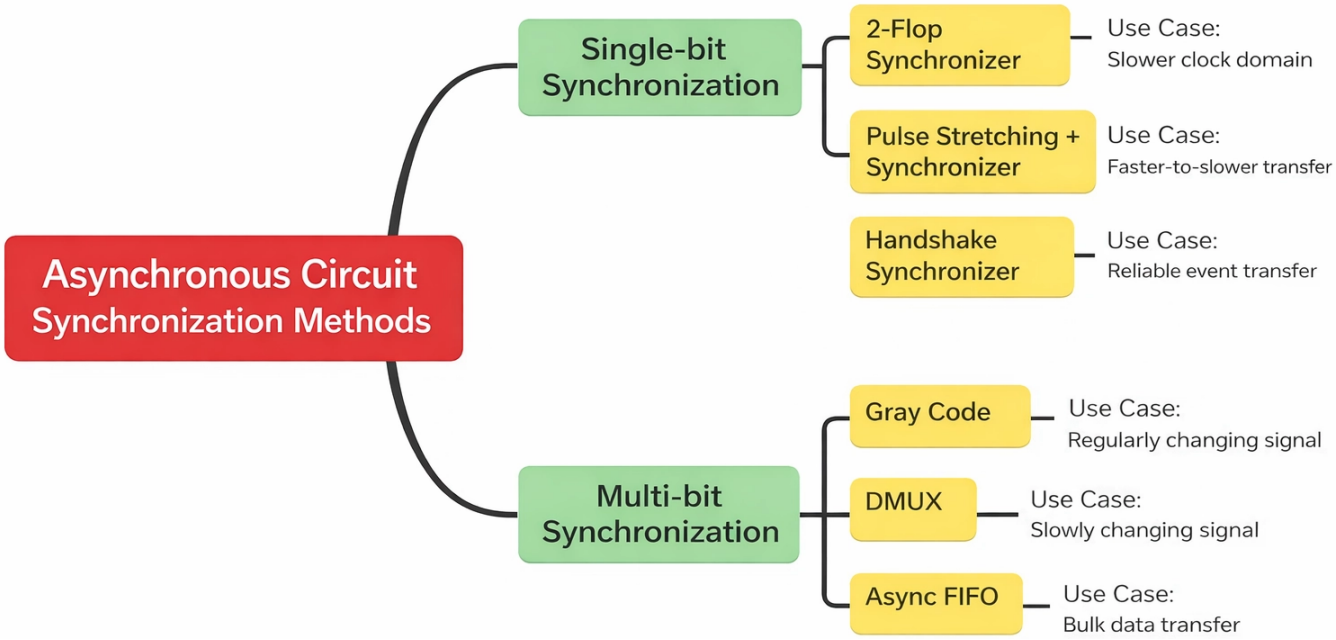
Asynchronous assertion, synchronous deassertion

```
always @ ( posedge clk or negedge rst_n ) begin
 if ( ~rst_n ) {sync_dff_1, sync_dff_0} <= 2'b0;
 else {sync_dff_1, sync_dff_0} <= {sync_dff_0, 1'b1};
end
```



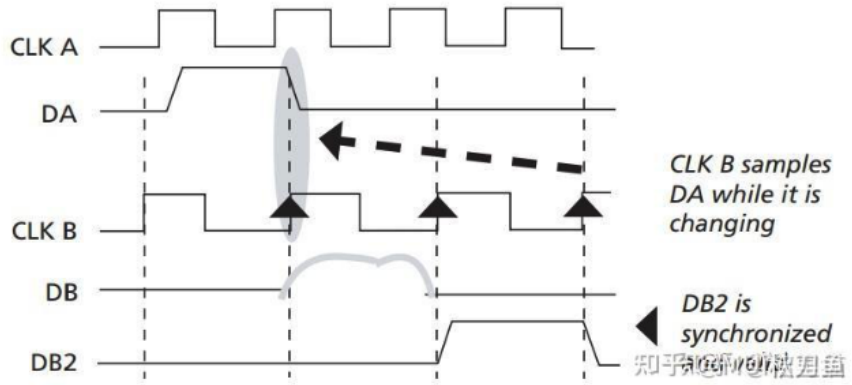
Synchronisation

https://zhuanlan.zhihu.com/p/1898391058276161024


Asynchronous Circuit
Synchronization Methods
Single-bit
Synchronization
2-Flop
Synchronizer
Use Case:
Slower clock domain
Pulse Stretching +
Synchronizer
Use Case:
Faster-to-slower transfer
Handshake
Synchronizer
Use Case:
Reliable event transfer
Multi-bit
Synchronization
Gray Code
Use Case:
Regularly changing signal
DMUX
Use Case:
Slowly changing signal
Async FIFO
Use Case:
Bulk data transfer

Single-bit

1. 2FF Synchronizer (**FAST sampling SLOW**)
    - f_{clkA} needs to be at least 1.5x f_{clkB}. Value should hold for at least 3 clock edges.
    - The source signal cannot be output from combinational logic!!



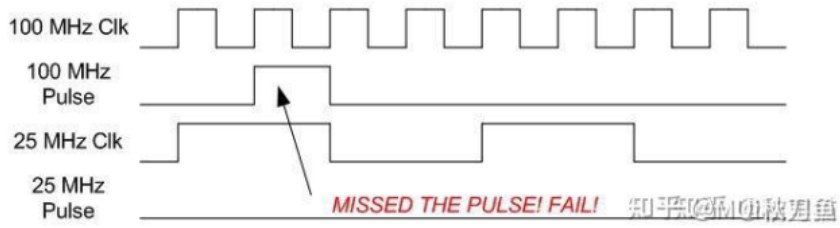
Example RTL

```
module sync_2ff (
 input logic clk,
 input logic rst_n,
 input logic async_in,
 output logic sync_out
);
 logic sync_ff1, sync_ff2;
 always_ff @(posedge clk or negedge rst_n) begin
 if (!rst_n) begin
 sync_ff1 <= 1'b0;
 sync_ff2 <= 1'b0;
 end else begin
 sync_ff1 <= async_in;
 sync_ff2 <= sync_ff1;
 end
 end
 assign sync_out = sync_ff2;
endmodule
```

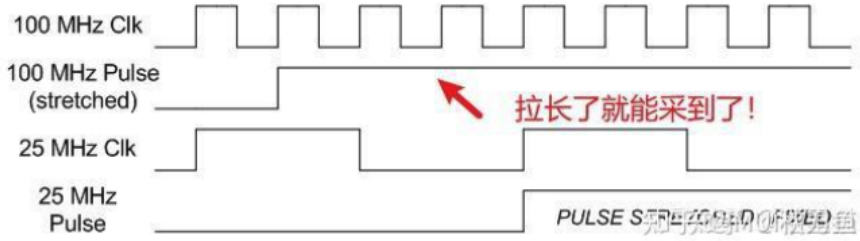
1. 2FF Synchronizer (**SLOW sampling FAST**)

   Same as above.

1. Stretch signal



比如，上面这个脉冲采不到。不如直接把它个拉长一些。



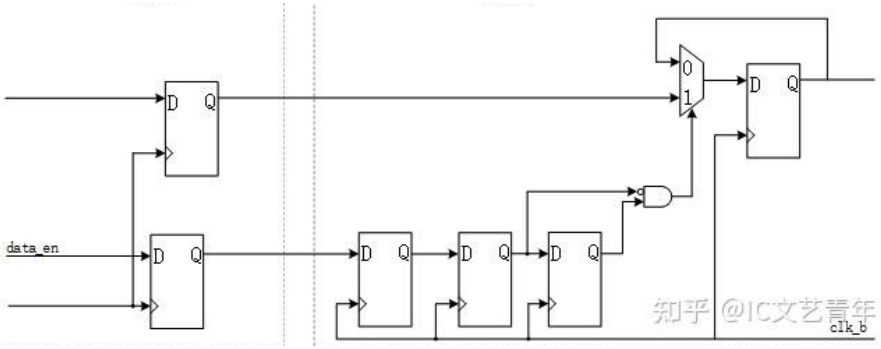
1. Handshake
    - Send signal to slow clock
    - Slow clock send back same signal
    - Use the same signal to validate data from faster clock

Pulse Synchronizer

Multi-bit

1. Gray Code
1. DMUX

    The following shows multiple flip flops. Used when Source frequency is much higher than Destination frequency.



```
// Source domain: hold data stable and assert enable
always_ff @(posedge src_clk or negedge src_rst_n) begin
 if (!src_rst_n) begin
 data_bus <= '0;
 data_en <= 1'b0;
 end else begin
 if (send) begin
 data_bus <= src_payload;
 data_en <= 1'b1;
 end
 end
end
// Destination domain: synchronize enable
always_ff @(posedge dst_clk or negedge dst_rst_n) begin
 if (!dst_rst_n) begin
 data_en_sync1 <= 1'b0;
 data_en_sync2 <= 1'b0;
 end else begin
 data_en_sync1 <= data_en;
 data_en_sync2 <= data_en_sync1;
 end
end
// Destination domain: capture stable bus
always_ff @(posedge dst_clk or negedge dst_rst_n) begin
 if (!dst_rst_n) begin
 dst_payload <= '0;
 dst_valid <= 1'b0;
 end else begin
 if (data_en_sync2) begin
 dst_payload <= data_bus;
 dst_valid <= 1'b1;
 end else begin
 dst_valid <= 1'b0;
 end
 end
end
```

1. Async FIFO

System Verilog Notes

Datatype

Verilog `reg` can be synthesized to a net or register depending on usage, combinational/sequential.

- 2-state type initializes to 0
- From 4-state to 2-state → x or z is converted to 0, no warning

X and Z meaning

1. **Z — High Impedance**
   Means the signal is **electrically disconnected** — no driver is actively driving it.
1. **X — Unknown**

   Means the value is **indeterminate** — the simulator cannot determine if it's 0 or 1.

Connection datatype rules

1. Instance output ports → `net`

```
wire quotient
divider d1(quotient, divisor, dividend)
```

1. Assigned from procedural block → `reg`

```
module mone(output reg y_out,
 input wire a_in, b_in);
 always @(a_in or b_in)
 assign y_out = a_in && b_in;
endmodule
```

1. Assigned from continuous `assign` → `net`

```
module mone(output wire y_out,
 input wire a_in, b_in);
 assign y_out = a_in || b_in
endmodule
```

1. Only net types (`wire`) can have multiple drivers
1. `always_ff`, `always_comb`, `always_latch` allow only 1 driver to variables. `always @` can have multiple drivers to variables

Randomize

`$random`, `$urandom` and `$urandom_range`

Example:

```
bit [15:0] value;
std::randomize (value) with {value inside{[-3:5]};}; // -3 ~ 5
value = $urandom_range(0,7); // 0 ~ 7
value = $random % 60; // -59 ~ 59
```

Procedural statements / blocks

`timeunit` and `timeprecision`

The current time unit is defined using the following highest to lowest precedence:

1. The `timeunit` of the current interface, module, package or program definition.
1. The `timeunit` inherited from the enclosing interface or module definition.
1. The timescale (`` `timescale ``) directive of the current compilation unit.
1. The `timeunit` declaration of the current compilation unit.
1. The implementation-specific default time unit.

`foreach`

Visible only within block or any nested. Read-only loop variable.

```
int arr2d [7:0][2:0];
foreach (arr2d [k,l])
 arr2d[k][l] = k*l;
```

```
for (int k=7;k>=0;k--)
 for (int l=2;l>=0;l--)
 arr2d[k][l] = k*l
```

`do…while`

Executed once even if loop condition is false.

> Syntax
>
> ```
> do
>  statement_or_null
> while ( expression ) ;
> ```

`Case` synthesis

`(* synthesis parallel_case *)`

- Remove inferred priority logic
- Remove priority encoders from gate-level

`(* synthesis parallel_case, full_case *)`

- Tell synthesis to treat outputs as **don't care** assignments or all unspecified case choices
- Use `default` and avoid full case

`priority case`

Equivalent to `full_case` attribute, overlapping branches permitted.

```
priority case (fullc)
 0: op = a;
 0,1,2 : op = b; // Overlapping but OK
 3: op = c;
endcase
```

`unique case`

In contrast to priority, only 1 branch activated at single trigger.

```
priority case (fullc)
 0: op = a;
 0,1,2 : op = b; // Overlapping RUNTIME WARNING
 3: op = c;
endcase
```

`priority if`

Branch conditions chekced **sequentially** and executes first qualified.

> Can be made complete with `else ;`

`unique if`

Branch condition tested simultaneously. **ONE BRANCH MUST MATCH!**

`iff` qualifier

Qualifies a procedural event control.

```
always @(avec iff enable == 1)
 y <= avec;
```

```
always @(avec)
 if (enable)
 y <= avec;
```

> Typical Usage

```
// RTL
always @(posedge clk iff (gate == 1)
 or posedge rst)
 if (rst)
 data_out <= 8'h00;
 else
 data_out <= bvec;

//TB
foreach (payload [i])
 // send payload when not suspended
 @(negedge clk iff (!suspend))
 data_out <= payload[i];
```

Block assignments

`always_comb`

- Run once at time 0.
- Sensitivity list that includes every variable read by procedural block

`always @*`

- Can have timing / event controls
- Only sensitive to changes in the arguments of a called function

> ? Wildcard sensitivity list inside Verilog always block?
> always @* with time example?

`always_latch`

- Cannot have timing / event controls
- Run once at time 0.

`always_ff`

- Variables assigned in always_ff cannot be reassugned by another procedure.
- **ONE AND ONLY ONE** event control
- No block timing

Operators

Equivalence operators

- **== (equality)**: Returns true if operands are equal. If either operand contains x or z, the result is x (unknown).
- **==? (wildcard equality)**: Treats z and ? in the right operand as "don't care" values. Allows pattern matching where certain bits can be ignored.
- **=== (case equality)**: Returns true only if operands are identical, including x and z values. Always returns 0 or 1, never x. Used when you need to explicitly compare unknown or high-impedance values.
- **!=? (wildcard inequality)**: The inverse of ==?. Returns true if operands are not equal, treating z and ? in the right operand as "don't care" values.

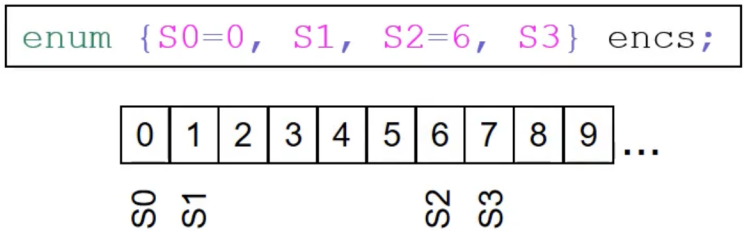
Array assignment

Some valid assignments: arr1 = ‘{0,1,2,3}; arr2 = ‘{3:1, default:0};

User-Defined Data Types

Enumerated

typedef enum {s0=0, s1, s2=6, s3}



Access: `first(), last(), next(N), prev(N), num(), name()`

Example: `op = op.first(); repeat(op.num())` Returns number of values in `op`

Package

To avoid type mismatch, if only declared in enum, won't compile.

```
package mytypes;
 typedef enum {start,done} mode_t;
 ...
endpackage : mytypes
// Call in module header
module mtwo import mytypes::*;(
 input logic [7:0] out,
 output mode_t mode);
)
```

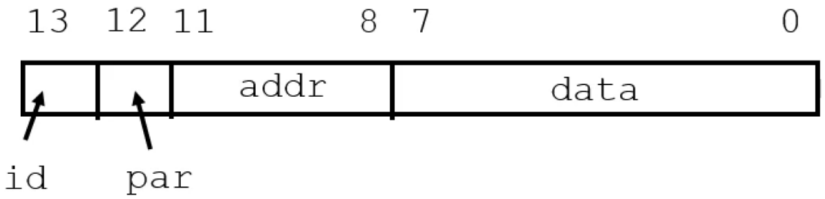
Struct

Use dot notation. Eg.

```
typedef struct{
 logic[7:0] data;
} frame_t
frame_t f1
data_in = f1.data
```

Struct packed

Can be treated as 1D vector.




```
typedef struct packed {
  logic       id, par;
  logic[3:0] addr;
  logic[7:0] data;
} frame_t;

frame_t f1, f2;

logic [3:0] addrbuf;
initial begin
  addrbuf = f1.addr;
  addrbuf = f1[11:8];
  f2 = f1 >> 2;

  f1 = 8'haa;      //14'h00aa
  f1 = 16'h5555;   //14'h1555
...
```

All fields are 4-state types

Same field

Padding

Truncation

Hierarchy and Connectivity

Port Connection

.name or .*(if all signals and port names match)

Using .name (explicit port connections):

```
adder add1 (
 .a(signal_a),
 .b(signal_b),
 .sum(result)
);
```

Using .* (automatic connections when names match):

```
// If module has ports: a, b, sum
// and current scope has signals: a, b, sum
adder add1 (.*);
adder add2 (.*, .sum(SUM)) //.* must come first
adder add3 (.*, .sum) // PROHIBITED
```

Import and Export

Import can be used anywhere **except Classes**

Preferred import in module header:

```
module mone import P1::*;( //Semicolon required
 input mode_t mode,
 ...);
...
endmodule
```

Static Arrays

Compilation Unit Scope (CUS)

Use `$unit::` to refer to CUS localparam

```
typedef enum {start, done} mode_t;
localparam eseg = 7'b1111100;
module mone (
 input mode_t mode,
 output logic [7:0] out
);
 // ...
endmodule : mone
module mtwo (
 input logic [7:0] out,
 output mode_t mode,
 output logic [6:0] oa, ob
);
 localparam eseg = 7'b0011111;
 // ...
 assign oa = eseg; // local
 assign ob = $unit::eseg; // CUS
endmodule : mtwo
```

Array Querying Statements

```
typedef logic [7:0] mem_t [1:1024];
mem_t mem;
$display($dimensions(mem)); // 2
$display($unpacked_dimensions(mem_t)); // 1
$display($left(mem)); // 1
$display($high(mem_t, 2)); // 7
$display($increment(mem)); // -1 (-1 if $left >= $right, else 1, check whether the array is incremental)
$display($size(mem)); // 1024
```

Packed and Unpacked Array

```
typedef logic [3:0][7:0] packed_array
typedef logic [3:0] unpacked_array [7:0]
```

Task and Function

Static and Automatic Subroutine

Task

`task automatic` creates copy of variables for each call, thus can be called concurrently.

`task` creates static subroutine

Issues

- Input changes are not detected
- Only the last output update is seen

Solutions

1. Variable access by side-effect

   Leave the signals you want to detect outside, do not pass in.

   ```
   logic req;
   logic ack;
   logic [7:0] data;
   task cpu_drive_side(input logic [7:0] write_data);
    #5 req = 1; // driving req directly
    wait(ack == 1); // reading ack directly
    #20 data = write_data; // driving data directly
    wait(ack == 0);
    #20 data = 8'hzz;
    req = 0;
   endtask
   cpu_drive_side(8'hff);
   ```

1. Argument passing by reference

   Create `task automatic` and put `ref` in in/out declaration.

```
task automatic cpu_driver_ref(
 input logic [7:0] write_data,
 ref logic data_read,
 ref logic data_valid,
 ref logic [7:0] cpu_data);

 #5 data_valid = 1; // writes through ref → external
variable becomes 1
 wait (data_read == 1); // waits on real external signal
 #20 cpu_data = write_data; // writes directly outside
 wait (data_read == 0); // again waits on live updates
 #20 cpu_data = 8'hzz; // immediate external update
 data_valid = 0; // external goes to 0
endtask : cpu_driver_ref
cpu_drive_side(8'hff, req, ack, data); // <--- Give the
reference
```

Use const ref to prevent writing argument passed by reference!

Void Function

`task/function void` declares task/function without returning values, eg. Print errors

```
function void func(input int errs);
...
endfunction
```

To disgard function returned values:

```
function bit func(...);
...
endfunction
initial begin
 void'(func(...));
end
```

Interface

Interfaces can also contain much more complex features, such as "assign" statements, subroutines, procedural blocks, assertions, etc.

Interfaces cannot contain other design element declarations, such as modules, configurations, or packages, containing module-specific constructs such as "specify" blocks.

Parameterize

```
interface fastbus #(DBUS = 32, ABUS = 8)(input clk);
```

Call in module:

```
module test;
 fastbus #(8) bus8x8(clk);
```

```
interface slowbus;
 parameter WIDTH = 16
```

Call in module:

```
module testl;
 slowbus #(.WIDTH(8)) bus2();
```

Verification

String Operations

- **len()**: Returns the length of the string

```
string s = "hello";
int length = s.len(); // length = 5
```

- **putc()**: Replaces a character at a specific index

```
string s = "hello";
s.putc(0, "H"); // s = "Hello"
```

- **getc()**: Returns the character at a specific index as a byte

```
string s = "hello";
byte c = s.getc(1); // c = 'e' (101)
```

- **toupper()**: Converts all characters in the string to uppercase

```
string s = "hello";
s = s.toupper(); // s = "HELLO"
```

- **tolower()**: Converts all characters in the string to lowercase

```
string s = "HELLO";
s = s.tolower(); // s = "hello"
```

- **compare()**: Compares two strings lexicographically (returns 0 if equal, <0 if less, >0 if greater)

```
string s1 = "abc", s2 = "abd";
int result = s1.compare(s2); // result &lt; 0
```

- **icompare()**: Case-insensitive string comparison

```
string s1 = "Hello", s2 = "hello";
int result = s1.icompare(s2); // result = 0
```

- **substr()**: Extracts a substring from start index to end index

```
string s = "hello world";
string sub = s.substr(0, 4); // sub = "hello"
```

- **atoi()**: Converts string to integer

```
string s = "123";
int num = s.atoi(); // num = 123
```

- **atohex()**: Converts hexadecimal string to integer

```
string s = "1A";
int num = s.atohex(); // num = 26
```

- **atooct()**: Converts octal string to integer

```
string s = "17";
int num = s.atooct(); // num = 15
```

- **atobin()**: Converts binary string to integer

```
string s = "1010";
int num = s.atobin(); // num = 10
```

- **atoreal()**: Converts string to real number

```
string s = "3.14";
real num = s.atoreal(); // num = 3.14
```

- **itoa()**: Converts integer to string

```
int num = 123;
string s = string'(num); // s = "123" (or use $sformatf)
```

- **hextoa()**: Converts integer to hexadecimal string

```
int num = 255;
string s = $sformatf("%h", num); // s = "ff"
```

- **octtoa()**: Converts integer to octal string

```
int num = 15;
string s = $sformatf("%o", num); // s = "17"
```

- **bintoa()**: Converts integer to binary string

```
int num = 10;
string s = $sformatf("%b", num); // s = "1010"
```

- **realtoa()**: Converts real number to string

```
real num = 3.14;
string s = $sformatf("%f", num); // s = "3.140000"
```

Severity Levels

- `$fatal`: Terminates simulation immediately with an error code

```
$fatal(1, "Critical error: simulation cannot continue");
```

- `$error`: Reports a runtime error but continues simulation

```
$error("Invalid data received: %h", data);
```

- `$warning`: Reports a warning message without affecting simulation

```
$warning("Buffer nearly full: %0d/%0d", count, max);
```

- `$info`: Provides informational messages for debugging

```
$info("Transaction completed at time %0t", $time);
```

Immediate assertion

Instantaneous Bool check

```
assert (ce)
```

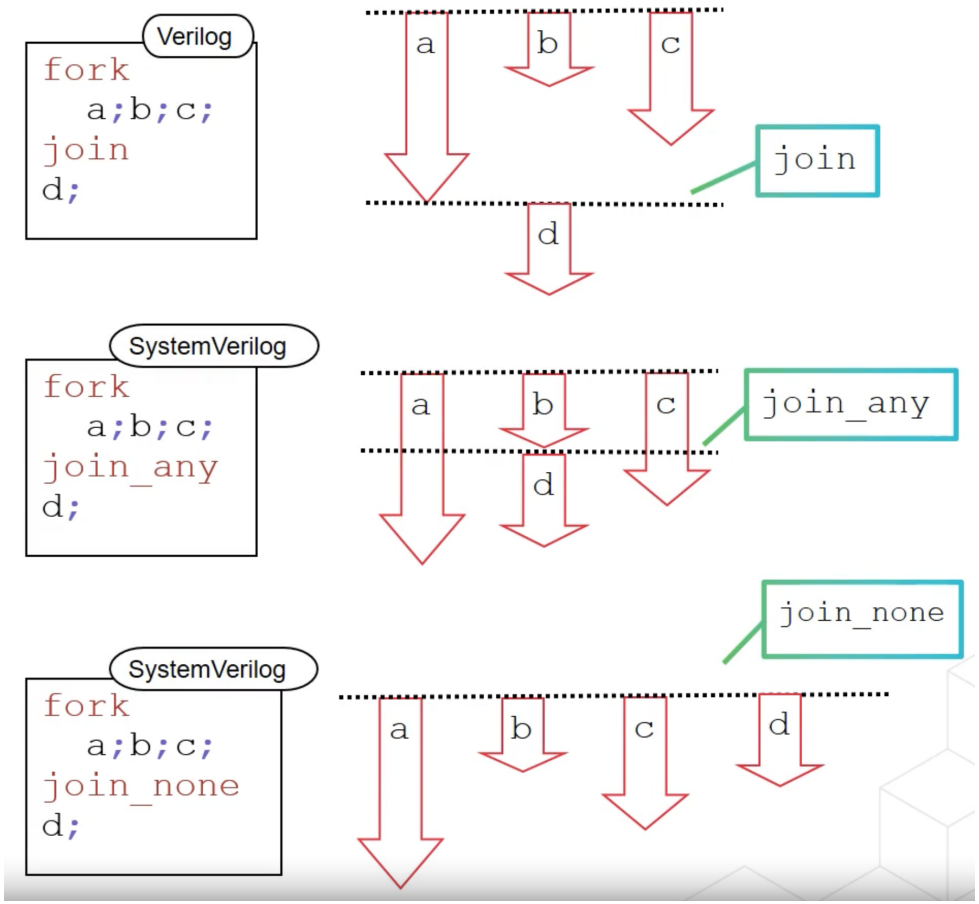
Concurrent assertion

Can span multiple cycles, describe behavior over time

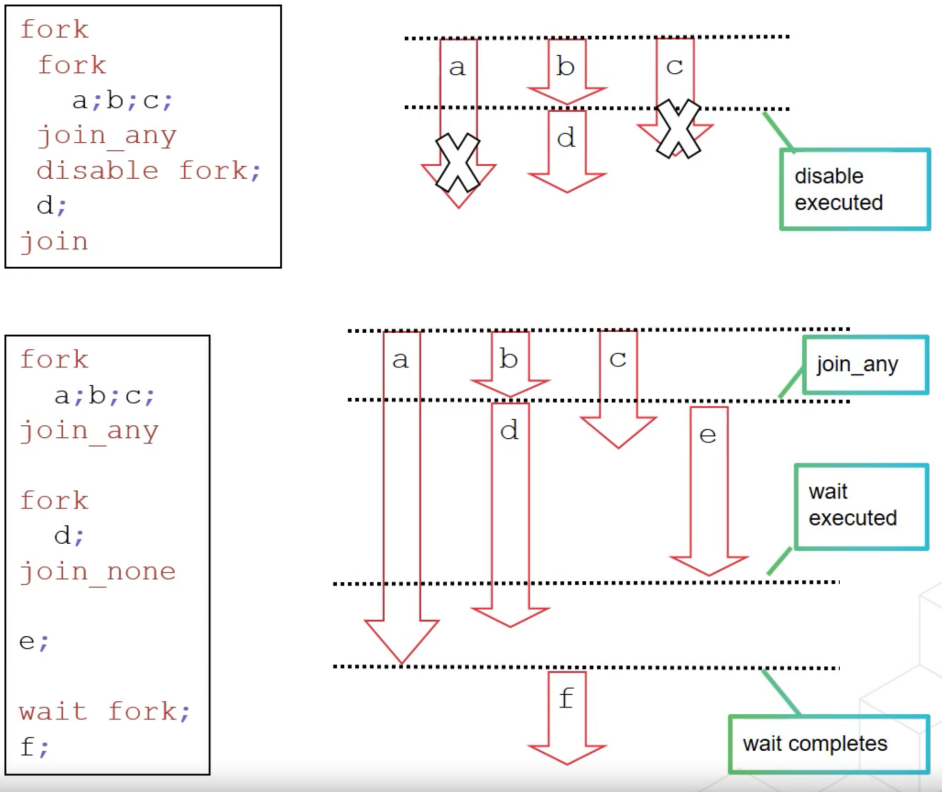
```
assert property(
 @(posedge clk)
 !ce ##1 ce |-> cd[*16] ##1 !ce
);
```

Fork-join

1. `join_any` completes the fork as soon as any of the blocks completes
1. `join_none` completes the fork immediately



1. `disable_fork` terminate all active descendants of the current process.
1. `wait fork` stops further execution until all forked blocks complete



Clocking block

Already an instance, can be declared in interface, module and program. Cannot be declared in a package or CUS!

An example:

Provided flip_flop.sv

```
module flipflop (input logic clk, reset,
 input logic [7:0] qin,
 output logic [7:0] qout);
timeunit 1ns;
always @(posedge clk or posedge reset)
 if (reset)
 qout = '0;
 else
 qout = qin;
endmodule
```

Clocking block based testbench

```
`timescale 1ns/1ps
`define period 10
module testFlop;
 logic clk = 1'b0, reset;
 logic [7:0] qin, qout;
 logic [1:0] epoch = 0;
 logic [7:0] exp = 0;

 always #(`period/2) clk = ~clk;

 flipflop DUV(.*);

 default clocking cb @(posedge clk);
 default input #1 output #4;
 input qout;
 output qin, reset;
 endclocking

 task automatic expect_out (input logic [7:0] exp_i);
 assert (cb.qout === exp_i) begin
 $display("[PASS]Assertion successful at %tns: qout=%h, qin ==
%h", $time*0.001, cb.qout, qin);
 end else begin
 $fatal(1, "[FAIL]Assertion failed at %tns: qout=%h, qin == %h,
expected %h", $time*0.001, cb.qout, qin, exp_i);
 end
 endtask

//   initial begin
//     $timeformat ( -9, 1, " ns", 9 );
//     $monitor ( "time=%t exp=%h qin=%h qout=%h rst=%b",$time,
exp, qin, cb.qout, reset);
//   end

 initial begin

 cb.qin <= 0;
 cb.reset <= 1;
 ##2 cb.reset <= 0;

 for (epoch = 0; epoch < 3; epoch = epoch +1) begin
 $display("Starting test epoch %d", epoch);
```

Random Stimulus

Some declarations

```
typedef enum bit[2:0] {ADDI, SUBI, ANDI, XORI, JMP, JMPC, CALL} op_t;
typedef enum bit[1:0] {REG0, REG1, REG2, REG3} regs_t;
op_t opc;
regs_t regs;
logic [7:0] data;
int ok;
```

1. Relational: Data between 32 and 126

```
ok = randomize(data) with { data >= 32; data <= 126; };
```

1. List: `opc` in range `ADDI` to `ANDI` or `JMP` or `JMPC`

```
ok = randomize(opc) with { opc inside {[ADDI:ANDI], JMP, JMPC}; };
```

3. Distribution: `REG0-REG1` twice as likely as `REG2-REG3`

```
ok = randomize(regs) with { regs dist { [REG0:REG1] := 2,[REG2:REG3] := 1 };};
```

Conditional Constriants

```
typedef enum bit {SMALL, LARGE} mode_t;
mode_t mode;
ok = randomize(data) with
 {mode == SMALL -> data < 100;
 mode == LARGE -> data > 200;};
```

Random Case

use `randcase` , the numbers before are weights:

```
randcase
 20 : gen_a();
 30 : gen_b();
 10 : gen_c();
 5 : gen_d();
endcase
```

Class

External Method Declaration

Write `extern function` inside the class, then write the actual function in the same scope. Use function `myclass::<function name>` to refer to the class (scope resolution operator ::).

Constructor Method

The function new is pre-defined but can be subject to modifications:

Default constructor

```
class default;
 integer number;
endclass
defcon c1 = new;
```

Explicit constructor

```
class expcon;
 integer number;
 function new();
 number = 5;
 endfunction
endclass
expcon c2 = new;
// c2.number = 5
```

Explicit constructor with arguments

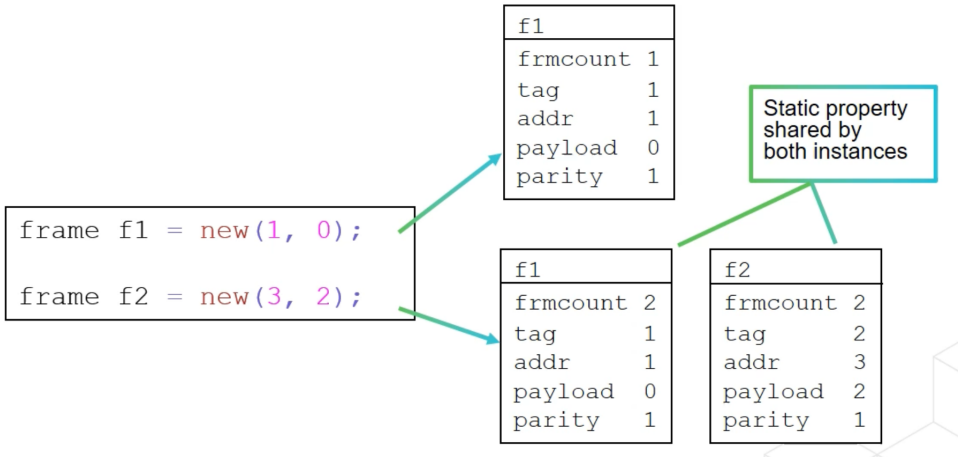
```
class argcon;
 integer number;
 function new(input int ai);
 number = ai;
 endfunction
endclass
argcon c3 = new(3);
// c3.number = 3
```

Static Class Properties

One instance of a static property exists for all instances of the same class type.

A static method of a class can access only the static members/properties of the class.

```
class frame;
 static int frmcount; // Static property
 int tag;
 logic [4:0] addr;
 logic [7:0] payload;
 logic parity;
 function new(input int add, dat);
 addr = add;
 payload = dat;
 genpar();
 frmcount++;
 tag = frmcount;
 endfunction
 function void genpar();
 parity = ^{addr, payload};
 endfunction
endclass
```



Class Inheritance

If the construct is modified, remember to call `super.new()` and pass in the expected inputs in superclass.

Example

```
// SUPERCLASS
class frame;
 logic [4:0] addr;
 logic [7:0] payload;
 bit parity;
 function new(input int add, dat);
 addr = add;
 payload = dat;
 genpar();
 endfunction
 function void genpar();
 parity = ^{addr, payload};
 endfunction
endclass
//INHERITED CLASS
class goodtagframe extends frame;
 static int frmcount;
 int tag;
 function new(input int add, dat);
 super.new(add, dat); // ✔ correct explicit call
 frmcount++;
 tag = frmcount;
 endfunction
endclass
```

Class-based Random Stimulus

```
class randclass;
 rand bit [1:0] p1;
 randc bit [7:0] p2;
 bit s1, s2;
 rand int p4;
 rand bit mode;
 constraint c1 {p1 != 2'b00;} // ";" after each expression not after the line
 constraint c2 {p2 >= 64; p2 <= 192;}
endclass
randclass myrand = new;
int ok, state;
initial begin
 myrand.rand_mode(0); // Disable all randomization
 myrand.p2.rand_mode(1);
 state = myrand.p2.rand_mode(); // 1
 ok = myrand.randomize(); // Randomize p1 using c1, p2 using c2
 state = myrand.s1.rand_mode(); // ERROR: s1 is not random variable
end
```

Constraints are inherited:

```
class cst1 extends randclass;
 constraint c1 {}
 constraint c3 {!(p2 inside [1, 7, [10:255]]);}
 constraint c4 {p4 dist {7:=5, [11:20]:=3, [26:30]:/1};}
 constraint c5 {mode -> p4 == 0;} // When mode is 1, p4 is zero

 rand randclass obj1; // if no rand, the entire obj1 is not
randomized
 function new();
 obj1 = new;
 endfunction

endclass
randclass myrand = new;
initial begin
 myrand.c3.contraint_mode(0); //Disabled c3
end
```

The `cst1` class overrides `c1` and added a new `c3` , which is not inside the range specified.

`c4` defines the weighted distribution:

Num	7	11:20	26:30
Weight	5	Each number: 3	Each Numb

Constraint `c5` may be unordered. We should randomize `mode` first, and then `p4` .

```
constraint c5 {mode -> p4 == 0; solve mode before p4}
```

> `randc` is always randomized first, then ordered `rand` , then unordered `rand` .

> `randc` has a maximum of 8 bits.

> `rand` argument hierarchically descends

Encapsulation

Both `local` and `protected` are not visible to application (outside of the class).

`local` - only visible inside the class

```
class errframe extends tagframe;
 local static int errcount;
...
endclass
```

This declaration makes `errcount` only visible inside `errframe`, and it keeps only 1 copy alone. So we can accumulate it.

Parameterized Classes

Define using `class <classname> #(parameter type <name> = <type>);`

Example

```
class stack #(parameter type sign_t = int);
 local sign_t data[100];
 static int depth;
 // Push task
 task push(input sign_t indata);
 ...
 endtask
 // Pop task
 task pop(output sign_t outdata);
 ...
 endtask
endclass
// ------------------------------
// Example instantiations
// ------------------------------
// int stack (default parameter)
stack intstack = new();
// 8-bit vector stack
stack #(logic [7:0]) bytestack = new();
```

Polymorphism

`protected` - Visible inside class and subclasses

```
class frame;
 protected bit parity;
...
endclass
```

This declaration makes `parity` to be visible to all subclasses, subclasses can make edits by simply using the variable, do not require `frame::` resolution operator.

Copying Parent Instance to a Sub-class Handle

Using `$cast(destination, source)`, we can use subclass resources while having a superclass handle.

Example

Class declaration

```
class frame;
 ...
 function void iam();
 $display ("frame");
 endfunction
 ...
endclass
class tagframe extends frame;
 ...
 function void iam();
 $display ("tagframe");
 endfunction
endclass
```

Application

```
frame f1;
tagframe t1 = new(...);
tagframe t2;
initial begin
 f1 = t1;
 f1.iam(); // frame
 $cast(t2, f1);
 t2.iam(); // tagframe
```

Virtual Methods

Virtual allow access to methods of sub-class instance while held in a parent class handle.

It binds the functions during runtime, it is resolved according to the contents of a handle, not the type of the handle.

Only methods can be virtual, not properties! Can only get sub-class properties via virtual method (`get_property` function)

```
class base;
 virtual function void iam();
 $display ("Base");
 endfunction
endclass
class parent extends base;
 virtual function void iam(); // virtual keyword optional
 $display ("Parent");
 endfunction
endclass
class child extends parent;
 virtual function void iam(); // virtual keyword optional
 $display ("Child");
 endfunction
endclass
```

```
base b1;
parent p1 = new();
child c1 = new();
initial begin
 b1 = p1;
 b1.iam(); //Parent
 p1 = c1;
 p1.iam(); //Child
 ...
```

The compiler **ALWAYS** looks at handle type first, then look at instance content if there is any.

Example

```
class base;
 virtual function void one();
 $display("base1");
 endfunction
 function void two();
 $display("base2");
 endfunction
 virtual function void three(
);
 $display("base3");
 endfunction
endclass
class parent extends base;
 function void one();
 $display("parent1");
 endfunction
 function void two();
 $display("parent2");
 endfunction
 virtual function void four()
;
 $display("parent4");
 endfunction
endclass
```

```
base b;
parent p = new();
initial begin
 b = p; // b is the handle, so compiler looks at
 // what's inside b first, then if there is a virtual
 // method, the call will be re-directed to subclass
 b.one(); ------> "parent1"
 b.two(); ------> "base2"
 b.three(); ------> "base3" since three() is not defined in parent, the method is inherited fro
m base.
 b.four(); ------> Compilation Error, not defined in base handle.
end
```

Virtual Interface

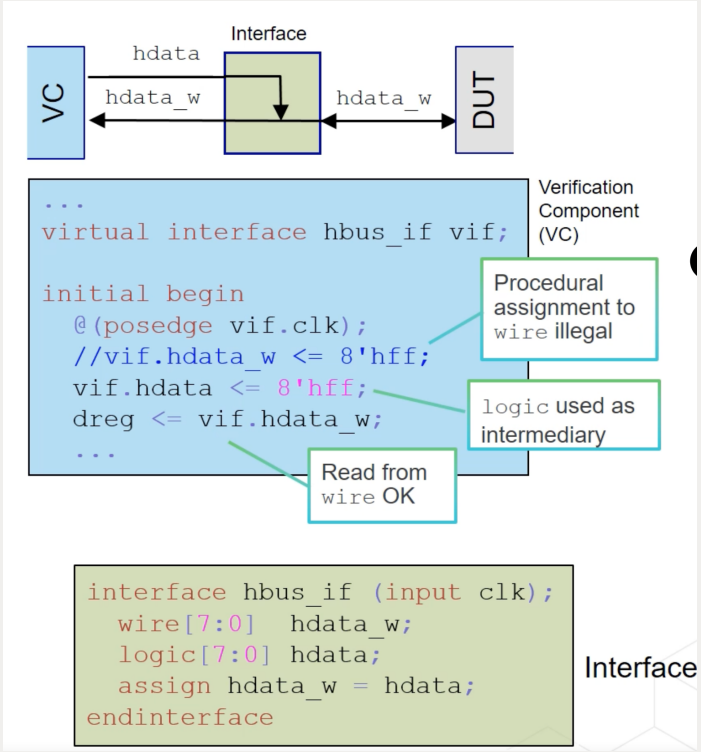
Avoid tying static interface instance names.

Problem

Interface signals can only be driven via a virtual interface using a procedur
assignment

1. Create a `wire` as intermediate.

   Example



```
interface myif (input clk);
 logic [7:0] data;
 // ... more signals if needed
endinterface
class myvc;
 virtual myif vif;
 function new (virtual myif aif);
 vif = aif;
 endfunction

 task write_data(input int datin);
 @(posedge vif.clk); // wait for clock in the interface
 vif.data = datin; // drive interface signal
 endtask
endclass
myif bus1(clk), bus2(clk);
myvc c1, c2;
initial begin
 c1 = new(bus1); // c1 connected to bus1
 c1.write_data(5); // drives bus1.data
 c2 = new(bus2); // c2 connected to bus2
 c2.write_data(8); // drives bus2.data
end
```

Coverage

Data and Control Oriented Functional Coverage

Covergroup (Data-oriented)

Example

```
covergroup cg @(posedge clk);
 cpa: coverpoint addr {
 bins low = { [0:'h0F], 19 };
 bins mid[] = { 16, 17, 18 };
 bins high = { ['h14:'hFF] };
 }
 addrxvalid : cross cpa, valid;
endgroup
cg cg1 = new();
```

Property (SystemVerilog Assertions for Control-oriented)

Example

```
property req_gnt (cyc);
 @(posedge clk)
 req ##[cyc] gnt;
endproperty
Covreqgnt_3: cover property (req_gnt(3));
Covreqgnt_4: cover property (req_gnt(4));
Covreqgnt_5: cover property (req_gnt(5));
```

Similar to a class object. `bins` are not holding the values, they are counters.

Covergroup

Can be declared in package/interface/module/program/class.

Contains:

1. Sampling event.
1. `coverpoints` for each variable to track. Can also track integral expressions.

Cover Point

A user-defined item in a coverage model specifying an expression to cover and optionally a condition guarding its sampling.

Each coverpoint generates a series of counters (bins). By default there is a corresponding bin for each coverpoint.

Explicit Bins

`illegal_bins` take precedence, it overrides other bins.

```
c1: coverpoint var {bins V = {1,2,5};}
logic [3:0] var1;
ce: coverpoint var1 {
 // 1 bin for illegal values {0, 15}
 illegal_bins a = { 0, 15 };
 // 1 bin for ignored values {13, 14}
 // (value 15 is illegal)
 ignore_bins b = { [13:15] }
 // 1 bin for {2, 3}
 bins c = { 2, 3 };
 // 4 bins e[0], e[1], e[2], e[6]
 bins e[] = { [0:2], 2, 6 };
 // 2 bins - d[0] = {9,10,11,9}
 //        - d[1] = {12}
 bins d[2] = { [9:11], 9, [12:15] };
 // 1 bin for {4,5,7,8}
 bins f = default;
}
```

Cover Cross

Create a matrix by `cross <bin1>, <bin2>, …;`

`binsof` selects specific bins for a coverpoint.

`intersect` filters bin selection to specified value range.

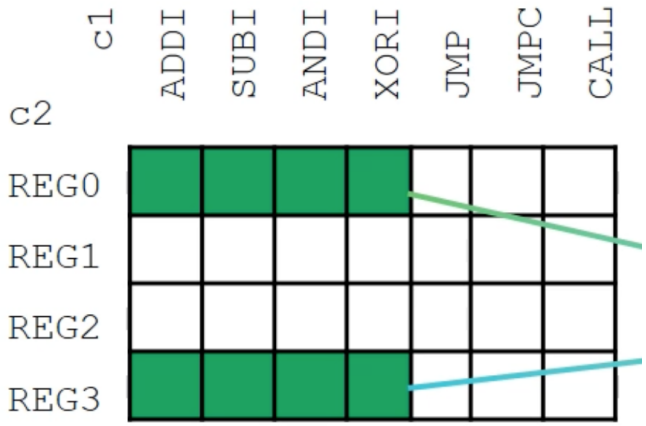
Example

```
typedef enum bit[2:0] {
 ADDI, SUBI, ANDI, XORI, JMP, JMPC, CALL
} op_t;
typedef enum bit[1:0] {
 REG0, REG1, REG2, REG3
} regs_t;
covergroup cg @(posedge clk);
 c1: coverpoint opc;
 c2: coverpoint regs;
 opxr1 : cross c1, c2 {
 bins x1 = binsof(c1) intersect { [ADDI:XORI] } && binsof(c2) intersect {
REG0, REG3 };
 }

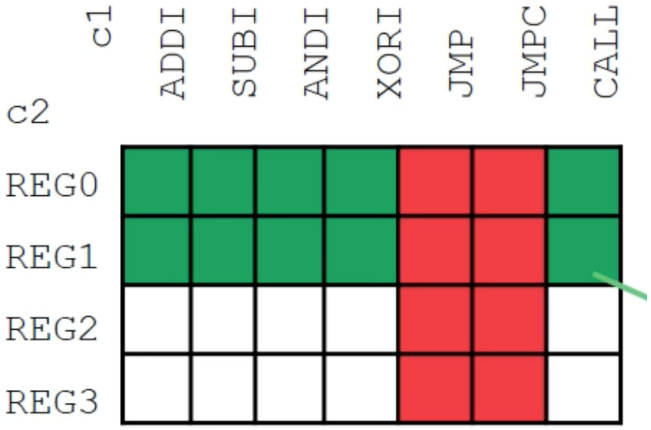
 opxr2: cross c1, c2 {
 bins x2 = !binsof(c2) intersect {[REG2:REG3]};
 ignore_bins x3 = binsof(c1) intersect {[JMP:JMPC]};
 }

endgroup
```

`opxr1` Diagram



`opxr2` Diagram



Transition Cover

Basic construct:

```
c1: coverpoint opc{bins trans1 = (A => B);}
 \_____/
```

1 transition	sequence	2 transitions	consecutive repetition
`A => B`	`A => B => C`	`A => B, C`	`A[*3]`

Options

Declaring an option by `type_option` or `option` with scope resolution operator `::`.

Example

Type Option

```
int a, b;
covergroup cg1 @(posedge clk);
 c1: coverpoint a {type_option.comment = "a";}
 c2: coverpoint b;
endgroup : cg1
cg1::type_option.comment = "ab";
```

Instance Option

```
int a, b;
covergroup cg1 @(posedge clk);
 c1: coverpoint a {option.auto_bin_max = 10;}
 c2: coverpoint b;
endgroup : cg1
cg1 one = new;
one.c2.option.auto_bin_max = 256;
```

Type-Specific `type_options`

Type	Identifier	Default Value	cg	cp	cc	Description
int	weight	1	✓	✓	✓	Weight of this element for calculation of covergroup or overall coverage
int	goal	100	✓	✓	✓	Target goal
string	comment	“”	✓	✓	✓	Comment to include in coverage report
bit	strobe	0	✓			Whether to defer samples to end of time slot

Instance-Specific `option`

Type	Identifier	Default Value	cg	cp	cc	Description
string	name	unique	✓			Instance name
int	weight	1	✓	✓	✓	Weight of this element for calculation of covergroup or overall coverage
int	goal	90	✓	✓	✓	Target goal
string	comment	""	✓	✓	✓	Comment to include in covera report
int	at_least	1	✓	✓	✓	Target count to consider bin as “covered”
int	auto_bin_max	64	✓	✓		Max number of auto bins
int	cross_num_print_missing	0	✓		✓	Max number of uncovered cross bins to report
bit	detect_overlap	0	✓	✓		If true issue warning for value overlapping bins
bit	per_instance	0	✓			If true track coverage for eac instance

Covergroup Methods

Method	cg	cp	cc	Description
function void sample()	✓			Force an immediate sample
function real get_coverage ([ref int cov, ref int tot])	✓	✓	✓	Get coverage for type – this is a static method
function real get_inst_coverage ([ref int cov, ref int tot])	✓	✓	✓	Get coverage for instance
function void set_inst_name(string)	✓			Set the instance name
function void start()	✓	✓	✓	Start collecting coverage
function void stop()	✓	✓	✓	Stop collecting coverage

QDA

Dynamic Array

Use `new` to create array or change size:

1. `new[size]`
1. `new[size](array)` Keeps original array

    Example

```
parameter num_vectors = 32;
logic [7:0] dyn[];
bit [4:0] addr;
logic [7:0] wdata, rdata;
initial begin
 dyn = new[num_vectors];
 $display("dyn size:%d", dyn.size());

 for (int i=0; i<num_vectors; i++) begin
 write_mem(i, i+5);
 dyn[i] = i+5;
 end

 for (int i=0; i<num_vectors; i++) begin
 read_mem(i, rdata);
 if (rdata !== dyn[i])
 $display("Error addr:%h, Wrote:%h, Read:%h", i, dyn[i], rdata);
 end
 dyn.delete();
 $display("dyn size:%d", dyn.size());
end
```

1.

Associative Array

Declaration example: `logic [7:0] array [int/string/…]`

Methods Available

Method	Description	Syntax
`num()` or `size()`	Returns the number of entries in an associative array	`function int num();` `function int size();`
`delete()`	Removes a single entry if an index is specified Removes all entries if no index specified	`function void delete(input index);`
`exists()`	Returns 1 if an element exists at the indexed location, otherwise returns 0	`function int exists(input index);`
`first()`	Assigns the value of the first (smallest) index in the array to index Returns 0 if array empty, otherwise 1	`function int first(ref index);`
`last()`	Assigns the value of the last (largest) index of the array to index Returns 0 if array empty, otherwise 1	`function int last(ref index);`
`next()`	Assigns next value to index Returns 1 if value exists, otherwise 0	`function int next(ref index);`
`prev()`	Assigns previous value to index Returns 1 if value exists, otherwise 0	`function int prev(ref index);`

Queue

Declare with `$` Sign

Declaration Example

```
integer q_integer[$]; // queue of integers
logic [15:0] q_logic[$]; // queue of 16-bit logic
int q_int[$:2000]; // queue of int - max size of 2000
time q_time[$:10]; // queue of time - max size of 10
```

Methods Available

Method	Description	Syntax
`size()`	Returns the size of the queue	`function int size();`
`insert()`	Inserts an item at the index location	`function void insert(int index, qtype item);`
`delete()`	Removes item at the specified index	`function void delete(int index);`
`pop_front()`	Removes an item in front of the queue [0]	`function q_type pop_front();`
`pop_back()`	Removes an item from the end of the queue [$]	`function q_type pop_back();`
`push_front()`	Places an item at the front of the queue [0]	`function void push_front(q_type item);`
`push_back()`	Places an item at the back of the queue [$+1]	`function void push_back(q_type item);`

Queue Example

```
int q_int[$]; // queue of unlimited size
int data;
initial begin
 q_int.push_front(0); // {0}
 q_int.push_back(1); // {0,1}
 q_int.push_front(2); // {2,0,1}
 q_int.insert(1, 3); // {2,3,0,1}
 q_int.insert(3, 4); // {2,3,0,4,1}
 q_int.delete(2); // {2,3,4,1}
 q_int.insert(2,5); // {2,3,5,4,1}
 data = q_int.pop_back(); // {2,3,5,4} data = 1
 data = q_int.pop_front(); // {3,5,4} data = 2
 while (q_int.size() > 0) // checking queue size
 data = q_int.pop_back(); // loop executes 3 times
 q_int = {0,1,2,3,4,5}; // direct load of queue - size = 6
 data = q_int[3]; // direct read - data = 3
 q_int = {}; // delete with empty queue assignment
end
```

Array Manipulation

Array Locator Methods

Locator Method	Description
`find()`, `find_first()`, `find_last()`	Find element(s) satisfying relational expression with expression is mandatory
`find_index()`, `find_first_index()`, `find_last_index()`	Find index(s) of elements satisfying relational expression with expression is mandatory
`min()`, `max()`, `unique()`	Find element(s) satisfying expression with expression is optional
`unique_index()`	Find indexes of elements having unique expression with expression is optional

```
string qstr[$], res[$]; int idx[$];
initial begin
 qstr = {"A","D","C","B","A","E","C"};
 res = qstr.find(i) with (i>="D"); // {"D","E"}
 res = qstr.max(); // {"E"}
 idx = qstr.find_last_index with (item>"C"); // {5}
 res = qstr.unique(); // {"A","D","C","B","E"}
```

Array Ordering Methods

Ordering Method	Description
`reverse()`	Reverse array elements Specifying with is an error
`sort()`	Sort array elements (ascending)
`rsort()`	Sort array elements (descending)
`shuffle()`	Randomize array element order Specifying with is an error

```
string qstr[$];
initial begin
 qstr = {"A","D","C","B","E"};
 qstr.sort; // {"A","B","C","D","E"}
 qstr.reverse; // {"E","D","C","B","A"}
 qstr.shuffle; // {"D","C","E","A","B"}
```

Array Reduction Methods

Method	Description
`sum()`	Sum of expressions
`product()`	Product of expressions
`and()`	Conjunction of expressions (bitwise)
`or()`	Disjunction of expressions (bitwise)
`xor()`	Parity of expressions (bitwise)

```
logic [7:0] narr [3:0] = '{8'ha, 8'h0, 8'hf, 8'h5};
int ia2d[2][2] = '{default:2};
int j;
...
j = narr.xor; // 0
j = narr.sum; // 1e
ia2d = '{'{1,2},'{3,4}};
j = ia2d.sum with (item.product); // 14 ((4*3)+(1*2))
```

Assertion

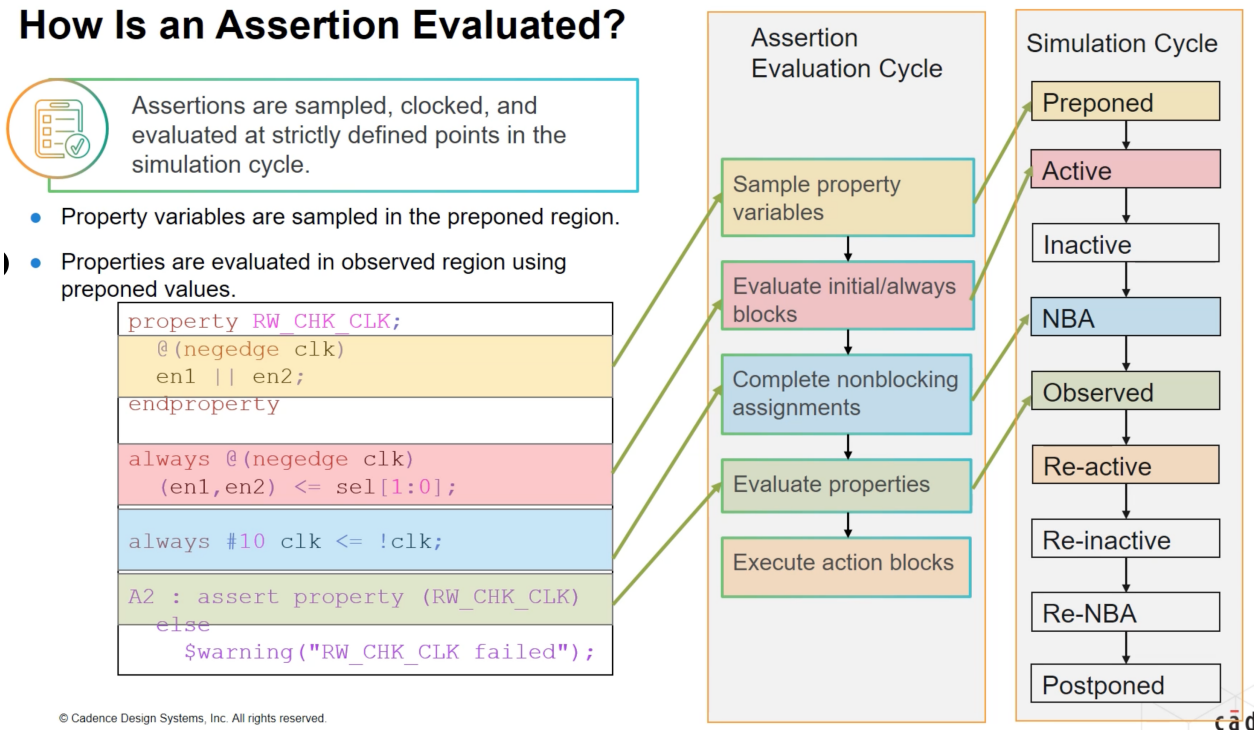
Naming and Declaration

There are two ways

```
ASSERT1 : assert property (@(posedge clk) en1 || en2);
```

```
property PROP1;
 @(posedge clk) en1 || en2;
endproperty
ASSERT2: assert property (PROP1)
```

Evaluation



Binding

Allows one or more instantiations of assertion module, interface, program or checker to be specified without modifying the code of the target.

To bind every instantiation of a DUT module:

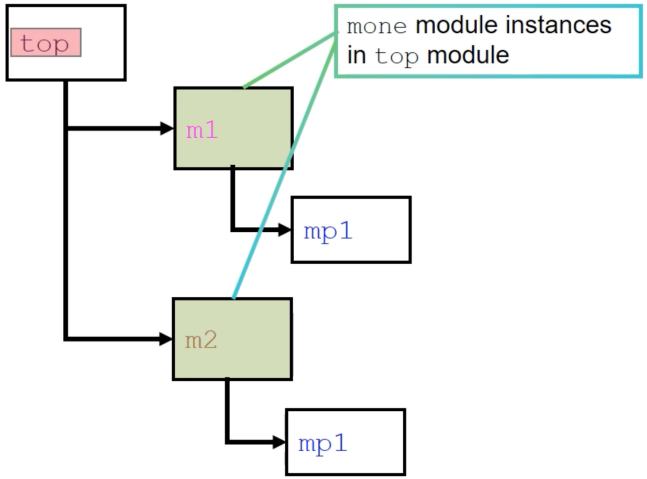
```
bind mone mone_prop mp1(req, ack, clk);
```

To bind to a specific instantiation of a DUT module:

```
bind mone:m1 mone_prop mp1(req, ack, clk);
```

```
module top;
 mone m1(clk, req1, ack1...);
 mone m2(clk, req2, ack2...);

 bind mone mone_prop mp1(req, ack, clk);
endmodule
```



Implication

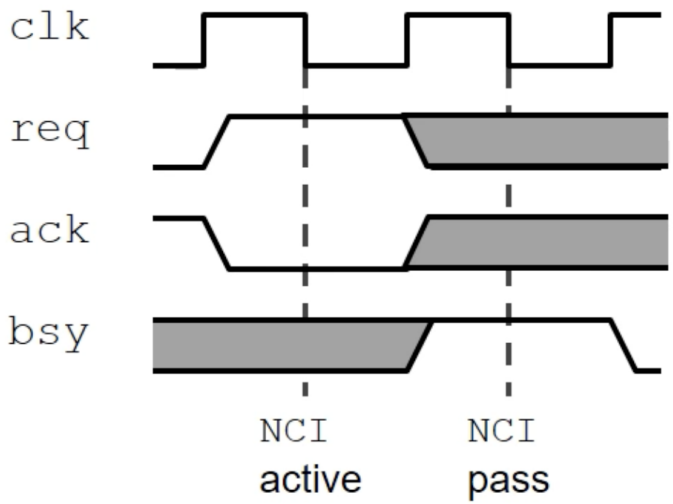
SAME CYCLE: Conditional operator `|->` means essentially:

1. If `expra` is true, then `exprb` much be checked
1. If `expra` is not true, `exprb` is not checked

> The `expra` is checked at every evaluation point, so checks can be overlapped.

NEXT CYCLE: Conditional operator `|=>` means essentially

```
property NCI;
  @(negedge clk)
    (req && !ack) |=> bsy;
endproperty
```



Consecutive Sequence Repetition: Use expr[*N].

Example

```
property SEQ_COUNT;
 @(negedge clk) !a[*4] |=> a; // @(negedge clk) !a ##1 !a
##1 !a ##1 !a |=> a;
endproperty
```

